CPE 334 · PROJECT DELIVERABLE 1

EquipFlow

Inventory and Borrowing System

Project No. 2 · Team TH-045_P2 · Sprint 1 engineering baseline and verification

Team member	GitHub	Student ID
Pawarisa Thongchua	itspxsh	67070501032
Norawit · full submission name to confirm	NxNxmm	To confirm before submission

Repository: NxNxmm/CPE334-Sec01-Team45

Review baseline: e7ee3849d617f2cb642e8e51e8fc807c91063f48

Release pending: expected tag `v0.1.0-sprint1`. The release PR, tag, and results from the submitted commit must be added after integration to main.

Contents

Part	Content	Pages
A	SRS and product scope · Stakeholders, actors, FR / BR / NFR and decisions	2–7
B	Feature inventory and traceability · F-01 to F-08, coverage and sprint allocation	8–13
C	System-level design · Architecture, data model, UI/API conventions and deployment plan	14–20
D	Three-sprint plan · Goals, dependencies, allocation and completion outcomes	21–23
E	Sprint 1 engineering contracts · F-01 to F-04, acceptance criteria, API/UI and test specification	24–43
F	Working Sprint 1 and verification · Execution, AC results, test excerpts, reviews and contributions	44–52

Submission checkpoint shown in LEB2: 15 November 2026, 23:59 ICT. This version is for teammate review.

Part A · SRS and product scope

Source: docs/srs.md

Software Requirements Specification (SRS)

CPE Equipment & Hardware Inventory and Borrowing System (EquipFlow)

Document ID: `SRS-01`
Version: `1.1.0`
Status: Approved **Course:** CPE 334 Software Engineering (Semester 1/2026)
Deliverable: Project Deliverable 1 — Part A
Due Date: 15 November 2026 at 23:59 (Aligned with LEB2)
Assigned Theme: Theme 2 — Inventory and Borrowing System

1. Stakeholder Problem & Context

1.1 Problem Statement

The Department of Computer Engineering maintains specialized educational hardware, testing instruments, IoT kits, and laboratory equipment (e.g., microcontrollers, oscilloscopes, FPGA boards, logic analyzers) for academic coursework, senior capstone projects, and research. Currently, the lending process relies on informal messaging, paper-based logbooks, or ad-hoc spreadsheets managed by lab custodians.

This causes critical operational problems:

1. **Lack of Real-Time Availability:** Students cannot determine whether equipment is currently on the shelf or in use without physically visiting the laboratory.

2. **Double-Allocation and Collisions:** Multiple students frequently request the same physical piece of equipment simultaneously, causing scheduling deadlocks and disputes.

3. **Loss of Custodial Custody and Accountability:** Custodians lack an authoritative, tamper-resistant system of record to log checkouts, enforce borrowing policies, monitor due dates, and hold borrowers accountable.

4. **No Role Separation:** Without verified identity and strict access controls, unauthorized users can manipulate reservation records or view other students' borrowing histories.

1.2 Chosen Context

EquipFlow is a web-based department inventory and lending management platform deployed for the CPE Department hardware laboratory. The system tracks individual serialized physical equipment units (assets) and governs the complete borrowing lifecycle between students/researchers (**Borrowers**) and lab technicians/equipment administrators (**Custodians**).

2. Actors & Responsibilities

Actor	Role Definition	Primary Responsibilities
Anonymous User (Visitor)	Unauthenticated visitor accessing the web portal.	• View landing/login page. • Authenticate using valid department credentials (login).
Borrower (Student / Faculty / Researcher)	Authenticated department member borrowing hardware.	• Browse catalog and check real-time availability. • Submit borrow requests for available items (within quota). • View own active borrow requests and personal loan history. • Cancel own pending requests before checkout (Sprint 2).
Custodian (Lab Administrator / Technician)	Authorized laboratory staff managing hardware assets and transactions.	• Maintain equipment catalog (create, edit, retire items - Sprint 2). • Review, approve, and fulfill pending borrow requests. • Record physical checkout (transitioning item from reserved to active loan). • Record direct walk-in checkout (Sprint 2). • Record return of equipment, assess physical condition, and close loans (Sprint 2). • Monitor overdue items and generate inventory audit summaries (Sprint 2 & 3).
System (Automated Agent)	Background scheduler and transactional boundary.	• Enforce exclusivity and atomic checkout locks. • Calculate and persist 7-day loan due dates upon checkout. • Evaluate overdue flags and trigger overdue state transitions (Sprint 2). • Maintain append-only audit trail for all state transitions (Sprint 3).

3. Product Scope & Explicit Exclusions

3.1 In-Scope Capabilities (Three-Sprint Roadmap)

• **Identity & Role-Based Access Control:** Secure authentication, bcrypt password hashing, JWT Bearer token lifecycle, server-side session invalidation on logout, and strict server-side authorization separating Borrower and Custodian operations.

• **Serialized Asset Inventory:** Tracking individual equipment units with unique asset tags, serial numbers, categories, models, conditions, and availability statuses.

• **Borrowing Lifecycle Management:** End-to-end state transitions: Item `AVAILABLE` → `RESERVED` (upon request) → `CHECKED_OUT` (upon custodian checkout) → `RETURNED` / `OVERDUE` (Sprint 2).

• **Exclusive Allocation & Concurrency Guard:** Atomic database transactions preventing double-booking, duplicate checkouts, or quota violations for physical assets.

• **Borrower Self-Service:** Personal dashboard showing current loans, pending requests, and recorded due dates.

• **Custodial Operations Desk:** Custodian dashboard for managing incoming requests, executing checkouts, inspecting item conditions, and processing returns.

• **Reporting & Auditing:** Inventory utilization, overdue tracking, and immutable transaction logs.

3.2 Explicit Exclusions (Out of Scope)

• **Financial Fines / Payment Gateway Integration:** The system will flag overdue items, but will not collect money, process credit cards, or compute monetary late fines.

• **Physical Barcode / RFID Hardware Scanners:** Barcode data strings can be typed or scanned via keyboard emulation; direct device drivers or proprietary RFID readers are excluded.

• **Multi-Department Multi-Tenancy:** The application is scoped strictly to a single department instance (CPE Lab); multi-tenant cloud partitioning is out of scope.

• **Automated External Communications:** No real SMS gateways, external SMTP servers, or LINE Notify webhooks are required; synthetic in-app notifications and status boards are used.

• **Consumable / Bulk Quantity Pooling in Sprint 1:** Sprint 1 focuses exclusively on uniquely serialized assets (discrete tracking) to enforce rigorous concurrency and state rules. Consumables (e.g., resistors, solder wire) are excluded from the core borrowing model.

• **Self-Registration & Password Recovery:** Seeded test accounts are provided for local sprint evaluations; public self-signup and email verification loops are excluded. Production cloud deployment will use externally provisioned credentials via environment variables.

4. Functional Requirements (FR)

4.1 Authentication & Session Management

• **FR-01 (User Login):** The system shall authenticate registered users using email and password, returning an authenticated JWT Bearer token upon credential validation.

• **FR-02 (Current User Identity):** The system shall provide endpoint `GET /api/v1/auth/me` returning the profile and role (`BORROWER` or `CUSTODIAN`) of the currently authenticated user.

• **FR-03 (User Logout & Per-Token Server-Side Revocation):** The system shall provide endpoint `POST /api/v1/auth/logout`. Upon invocation, the backend shall persist the Bearer token's unique identifier (`jti`) and expiry timestamp into the `RevokedToken` table. The authentication middleware shall reject any subsequent request presenting that specific token with HTTP 401 Unauthorized (`TOKEN_REVOKED`). Concurrent sessions for the same user (possessing different `jti` tokens) remain valid and unaffected.

• **FR-04 (Role-Based Authorization):** The system shall enforce server-side access control on all protected endpoints, denying unauthorized role actions with HTTP 403 Forbidden.

4.2 Catalog & Inventory Management

• **FR-05 (Catalog Browsing):** The system shall allow Borrowers and Custodians to view the catalog of equipment items via `GET /api/v1/items`, displaying asset tag, name, category, and real-time availability status.

• **FR-06 (Catalog Item Details):** The system shall allow users to inspect detailed specifications, item serial number, current condition, and borrowing restrictions of an item via `GET /api/v1/items/:id`.

• **FR-07 (Item Creation & Management - Sprint 2):** The system shall allow Custodians to register new physical equipment assets with unique asset tags, serial numbers, and initial condition via `POST /api/v1/custodian/items`.

• **FR-08 (Item Status Management - Sprint 2):** The system shall allow Custodians to update item statuses (e.g., mark an item `MAINTENANCE` or `RETIRED`) via `PATCH /api/v1/custodian/items/:id/status`.

4.3 Borrow Request & Reservation Workflow

• **FR-09 (Submit Borrow Request):** The system shall allow an authenticated Borrower to submit a borrow request for any available equipment item via `POST /api/v1/requests`. Submitting a request transitions the item status from `AVAILABLE` to `RESERVED` and creates a `BorrowRequest` in `PENDING` state.

• **FR-10 (View Personal Requests & Active Loans):** The system shall allow a Borrower to view all personal borrow requests and active loans via `GET /api/v1/requests/my`.

• **FR-11 (Data Ownership Isolation):** The system shall restrict Borrowers to viewing only their own requests/loans and forbid access to another user's private borrowing data (HTTP 403 or scoped query).

• **FR-12 (Cancel Pending Request - Sprint 2):** The system shall allow a Borrower to cancel their own request provided it is still in `PENDING` status via `POST /api/v1/requests/:id/cancel`, reverting the item status to `AVAILABLE`.

4.4 Custodian Triage & Checkout Workflow

• **FR-13 (Custodian Request Triage):** The system shall allow Custodians to view all pending borrow requests across all borrowers via `GET /api/v1/custodian/requests`, ordered chronologically.

• **FR-14 (Record Physical Checkout against Request):** The system shall allow a Custodian to record the physical checkout of an item against a matching pending request via `POST /api/v1/custodian/checkouts`. The operation validates that the item is currently `RESERVED` for this specific request, transitions the item to `CHECKED_OUT`, sets the request to `APPROVED`, creates an active `Loan` record, and automatically calculates and stores a 7-day `dueDate`.

• **FR-15 (Direct Walk-in Checkout - Sprint 2):** The system shall allow Custodians to execute an immediate checkout for an on-premise borrower for an `AVAILABLE` item without prior web reservation via `POST /api/v1/custodian/checkouts/walk-in`.

• **FR-16 (Reject Borrow Request - Sprint 2):** The system shall allow Custodians to reject a pending borrow request with a recorded rejection reason via `POST /api/v1/custodian/requests/:id/reject`, reverting the item status to `AVAILABLE`.

4.5 Return, Overdue & Lifecycle Management (Sprints 2 & 3)

• **FR-17 (Record Item Return - Sprint 2):** The system shall allow Custodians to process an equipment return via `POST /api/v1/custodian/returns`, recording return date, condition upon return, setting loan to `RETURNED`, and restoring item status to `AVAILABLE`.

• **FR-18 (Overdue Tracking & Badges - Sprint 2):** The system shall calculate whether active loans have passed their recorded `dueDate` (`dueDate < now()`), dynamically flag them as overdue, and render overdue badges on borrower and custodian views.

5. Business Rules (BR)

• **BR-01 (Availability & Checkout State Consistency):** A Borrower can only submit a borrow request if the item's status is strictly `AVAILABLE`. Upon successful request creation, the item status immediately transitions to `RESERVED`. A Custodian can check out an item if and only if: The item is currently in status `RESERVED` and matches the specific `PENDING` request being fulfilled, OR The item is currently in status `AVAILABLE` (for Sprint 2 walk-in checkouts). An item in status `CHECKED_OUT`, `MAINTENANCE`, `RETIRED`, or `RESERVED` for *another* request cannot be requested or checked out. Any attempt returns HTTP 400 Bad Request with code `ITEM_NOT_AVAILABLE`.

• **BR-02 (Exclusive Allocation & Duplicate / Conflict Rejection):** A physical serialized asset can have at most one active reservation or loan at any time. If a second request is submitted for an item that has transitioned to `RESERVED` or `CHECKED_OUT`, the request is rejected with HTTP 400 Bad Request / 409 Conflict. If concurrent checkout operations are submitted for the same item or request, the operations are executed within an atomic database transaction. The first transaction commits and transitions the item to `CHECKED_OUT`; the concurrent attempt detects the changed state, rolls back, and returns HTTP 409 Conflict with code `CONCURRENT_CHECKOUT_CONFLICT`.

• **BR-03 (Role Segregation on Checkout):** Only users with the `CUSTODIAN` role are authorized to record checkouts and update item availability. Borrowers are strictly forbidden from performing checkout operations, even for their own approved requests (HTTP 403 Forbidden).

• **BR-04 (Combined Active Quota per Borrower):** A Borrower's active quota is defined as:
Active Quota = Count(Pending Requests) + Count(Active Loans) A Borrower may have at most three (3) active items combined (Active Quota ≤ 3). Any request submission that would cause Active Quota > 3 is rejected immediately with HTTP 400 Bad Request (`EXCEEDS_ACTIVE_QUOTA`). If a borrower submits concurrent requests across different tabs/sessions when already near the limit, requests are evaluated inside a transaction with lock/count verification. The request that causes the count to exceed 3 is aborted and rejected with HTTP 409/400.

• **BR-05 (Default Loan Duration & Due Date):** In Sprint 1, every newly created loan record automatically calculates and persists `dueDate = checkedOutAt + 7 days`. In Sprint 2, the overdue processing engine evaluates this persisted `dueDate` against current system time to flag overdue loans.

• **BR-06 (Request Expiration - Sprint 2):** A pending borrow request that is not checked out within forty-eight (48) hours of submission shall automatically expire, setting request status to `EXPIRED` and reverting the item to `AVAILABLE`.

• **BR-07 (State Machine Immutability - Sprint 2):** Loan state transitions are strictly unidirectional: `PENDING` request → `ACTIVE` loan → `RETURNED`. A closed or returned loan cannot be reverted to `ACTIVE`.

• **BR-08 (Custodian Accountability):** Every checkout record must automatically capture the authenticated custodian's user ID as the accountable operator (`custodianId`), preventing anonymous or forged checkouts.

6. Non-Functional Requirements (NFR)

NFR ID	Category	Requirement Description	Measurable Target	Practical Verification Method	Allocated Sprint
NFR-01	**Security**	Passwords must never be stored or logged in plaintext. Sample credentials restricted to local dev.	Passwords hashed using bcrypt with salt rounds ≥ 10. Zero plaintext passwords in database or logs.	Direct database query inspection & unit test inspecting User table.	**Sprint 1**
NFR-02	**Security / Auth**	Unauthenticated, unauthorized, or revoked token requests blocked at backend.	100% of protected API endpoints return HTTP 401 (if unauthenticated/revoked) or HTTP 403 (if wrong role). No data leaked in error body.	Automated Supertest / API integration test suite.	**Sprint 1**
NFR-03	**Data Integrity**	Concurrency conflicts on the same physical asset must be rejected safely.	Zero duplicate active loans in database under simulated concurrent checkout requests.	Concurrent API stress integration test verifying second request returns 409 Conflict.	**Sprint 1**
NFR-04	**Performance**	API response latency for catalog browsing and checkout transactions under normal local load.	95th percentile response time < 200 ms for local API queries on seeded dataset (≥ 50 items).	Automated integration test benchmark timer.	**Sprint 2**
NFR-05	**Reliability & Persistence**	Data persistence across application restarts. Database setup reproducible via migrations.	Zero data loss of users, inventory, or active loans after server process restart. Migration commands execute cleanly.	Automated restart verification script and migration runner check.	**Sprint 1**
NFR-06	**Usability & Responsiveness**	UI layout must be fully functional across mobile (375px), tablet (768px), and desktop (≥ 1024px).	Zero horizontal scrolling or clipped action buttons across standard viewport breakpoints.	Playwright responsive viewport automated checks.	**Sprint 2**

7. Assumptions, Design Decisions & Resolved Questions

7.1 Key Assumptions

1. **Discrete Asset Model:** The department manages high-value equipment identified by unique asset tags (e.g., `CPE-OSC-001`, `CPE-RPI-004`). Tracking is unit-based rather than aggregate quantity pooling.

2. **Local Seed vs Cloud Provisioning:** Seeded accounts (`borrower@cpe.kmutt.ac.th`, `custodian@cpe.kmutt.ac.th`) with known passwords are used strictly in local development and automated CI. Cloud environments will provision accounts via external configuration and secure migrations.

3. **Single Physical Location:** All equipment is housed within the central CPE Laboratory; multi-building transfers are not modeled.

7.2 Design Decisions

• **Decision 01 (Per-Token JWT `jti` Revocation):** Authentication uses signed HTTP Bearer Tokens (`Authorization: Bearer <token>`) containing `{ userId, role, email, jti, exp }`. Logout (`FR-03`) persists the token's unique `jti` into a database `RevokedToken` table until its expiry. This guarantees deterministic cross-instance invalidation for the logged-out device while safely preserving a user's concurrent sessions across multiple devices.

• **Decision 02 (Relational Integrity with Prisma ORM):** Use relational database foreign keys (`userId`, `itemId`, `loanId`) with atomic transactions (`prisma.$transaction`) to enforce concurrency rules and prevent orphaned borrowing records.

• **Decision 03 (Two-Tier Role Hierarchy):** Role enum is strictly `BORROWER` and `CUSTODIAN`. Borrowers can request and view personal history; Custodians control catalog and checkouts.

7.3 Resolved Questions

• **Q1: Does a Borrower's request immediately change the item status?** *Resolution:* Yes. Submitting a request transitions the item from `AVAILABLE` to `RESERVED`. Custodian checkout validates that the item is `RESERVED` for that request and transitions it to `CHECKED_OUT`. If rejected or cancelled, it returns to `AVAILABLE`.

• **Q2: How do active loans and pending requests interact regarding the quota of 3?** *Resolution:* Both count equally toward the limit: Active Quota = Count(Pending Requests) + Count(Active Loans) ≤ 3. Submitting a request when active count is 3 is rejected with HTTP 400.

Part B · Feature inventory and traceability

Source: docs/features.md

Feature Inventory

CPE Equipment & Hardware Inventory and Borrowing System (EquipFlow)

Document ID: `FEAT-01`
Version: `1.1.0`
Status: Approved **Deliverable:** Project Deliverable 1 — Part B

1. Feature Overview & Lifecycle Allocation

The EquipFlow platform is partitioned into eight (8) discrete functional features spanning a three-sprint development trajectory. Every baseline requirement (FR, BR, and NFR) defined in the SRS is assigned to an authoritative feature.

Feature ID	Feature Name	Description	Target Sprint	Primary Actor	Dependencies
F-01	**Authentication, Session, and Role Identity**	User login, JWT Bearer token issuance, current user identity (`/api/v1/auth/me`), server-side logout invalidation (`/api/v1/auth/logout`), password hashing, and role route guards.	**Sprint 1**	All Actors	None
F-02	**Serialized Equipment Catalog**	Browsing and viewing serialized equipment assets (`/api/v1/items`), specifications, and reproducible seed data. Custodian catalog editing added in Sprint 2.	**Sprint 1** (Core) **Sprint 2** (Manage)	All Actors	`F-01`
F-03	**Borrower Request Submission & Personal History**	Student self-service interface to request available hardware (`POST /api/v1/requests`), enforce combined active quota (≤ 3), prevent cross-user leakage, and view personal requests/loans (`GET /api/v1/requests/my`).	**Sprint 1**	Borrower	`F-01`, `F-02`
F-04	**Custodian Counter Checkout & Exclusive Allocation**	Custodian triage interface (`GET /api/v1/custodian/requests`), verifying requests, executing checkout of reserved items (`POST /api/v1/custodian/checkouts`), saving 7-day due date, and preventing concurrent collisions. Walk-in checkouts in Sprint 2.	**Sprint 1** (Core) **Sprint 2** (Walk-in)	Custodian	`F-01`, `F-02`, `F-03`
F-05	**Equipment Return & Condition Assessment**	Custodian workflow to process equipment returns	**Sprint 2**	Custodian	`F-04`

Feature ID	Feature Name	Description	Target Sprint	Primary Actor	Dependencies
		(`POST /api/v1/custodian/returns`), inspect physical condition, record notes, close active loans, and restore asset availability.			
F-06	**Due Date Enforcement & Overdue Tracking**	Overdue processing engine evaluating persisted 7-day `dueDate`, dynamically flagging overdue accounts, and warning borrowers/custodians.	**Sprint 2**	All Actors	`F-04`
F-07	**Catalog Filtering, Full-Text Search & Category Navigation**	Rich search and filter capabilities across categories, tags, brands, and availability statuses with optimized query performance.	**Sprint 2**	All Actors	`F-02`
F-08	**Inventory Audit History & Custodial Reporting**	Append-only historical audit logging for all status changes, damage logging, utilization summaries, and compliance reporting.	**Sprint 3**	Custodian	`F-04`, `F-05`, `F-06`

2. Detailed Feature Specifications

F-01: Authentication, Session, and Role Identity

• **Scope in Sprint 1:** Seeded users (Borrowers and Custodians). Endpoint `POST /api/v1/auth/login` validating email and bcrypt password, issuing an HTTP Bearer JWT containing `{ userId, role, email, jti, exp }`. Endpoint `GET /api/v1/auth/me` returning the authenticated user profile and role. Endpoint `POST /api/v1/auth/logout` executing per-token server-side revocation: persists the token's unique `jti` and expiry into the database `RevokedToken` table. The `requireAuth` middleware rejects any request presenting a revoked `jti` with HTTP 401 Unauthorized (`TOKEN_REVOKED`), while concurrent sessions on other devices remain completely intact. Server-side route middleware blocking unauthenticated (401) and unauthorized role requests (403).

• **Mapped Requirements:** `FR-01`, `FR-02`, `FR-03`, `FR-04`, `NFR-01`, `NFR-02`.

• **Verification Approach:** Automated unit tests for bcrypt hashing; Supertest API integration tests for valid/invalid login, role route guards, and `TEST-AUTH-03` asserting that logging out Session 1 revokes Token 1 (returns 401) while concurrent Session 2 (Token 2) continues to access protected routes with HTTP 200.

F-02: Serialized Equipment Catalog

• **Scope in Sprint 1:** Read-only catalog endpoints `GET /api/v1/items` and `GET /api/v1/items/:id` showing asset tag, equipment name, model, category, serial number, and status (`AVAILABLE`, `RESERVED`, `CHECKED_OUT`, `MAINTENANCE`). Seeding script for reproducible demo data.

• **Scope in Sprint 2:** Custodian catalog administration (`POST /api/v1/custodian/items`, `PATCH /api/v1/custodian/items/:id/status` for marking maintenance/retired).

• **Mapped Requirements:** `FR-05`, `FR-06`, `FR-07`, `FR-08`, `BR-01`, `NFR-05`.

• **Verification Approach:** API endpoint tests for `/api/v1/items`; Playwright UI tests verifying catalog table rendering and availability badges.

F-03: Borrower Request Submission & Personal History

• **Scope in Sprint 1:** Borrower submits a borrow request via `POST /api/v1/requests` with payload `{ itemId: string, requestNote?: string }`. **Availability Validation:** Backend verifies item status is strictly `AVAILABLE` (`BR-01`). If already `RESERVED` or `CHECKED_OUT`, request is rejected with HTTP 400 (`ITEM_NOT_AVAILABLE`). **Quota Validation:** Backend verifies combined active count: Count(Pending Requests) + Count(Active Loans) < 3 (`BR-04`). If ≥ 3, rejected with HTTP 400 (`EXCEEDS_ACTIVE_QUOTA`). **Concurrency Protection:** Evaluation and status update to `RESERVED` execute within a database transaction; concurrent requests for the same item or exceeding quota are rejected with HTTP 409/400. **Personal History:** Endpoint `GET /api/v1/requests/my` returns only the authenticated borrower's own requests and active loans. Strict ownership check ensures Borrower A cannot access Borrower B's records (`FR-11`).

• **Scope in Sprint 2:** Request cancellation by borrower (`POST /api/v1/requests/:id/cancel`).

• **Mapped Requirements:** `FR-09`, `FR-10`, `FR-11`, `FR-12`, `BR-01`, `BR-02`, `BR-04`, `NFR-02`.

• **Verification Approach:** Unit tests for combined quota calculation; Supertest integration tests verifying cross-user isolation and request creation; Playwright end-to-end request journey.

F-04: Custodian Counter Checkout & Exclusive Allocation

• **Scope in Sprint 1:** Endpoint `GET /api/v1/custodian/requests` lists pending borrow requests. Custodian records checkout via `POST /api/v1/custodian/checkouts` with payload `{ requestId: string, checkoutNotes?: string }`. **State Validation:** Backend validates that the referenced request is `PENDING` and the associated item is currently `RESERVED` for this specific request (`BR-01`). **Atomic State Transition:** Executed inside `prisma.$transaction`: Update item status to `CHECKED_OUT`. Update request status to `APPROVED`. Create `Loan` record storing `checkedOutAt = now()`, `dueDate = now() + 7 days` (`BR-05`), and `custodianId = authenticatedUserId` (`BR-08`). **Conflict Prevention:** If a duplicate checkout attempt occurs, the atomic transaction detects the changed state and rejects the second call with HTTP 409 Conflict (`BR-02`, `NFR-03`). **Role Restriction:** Rejects non-custodian callers with HTTP 403 Forbidden (`BR-03`).

• **Scope in Sprint 2:** Direct walk-in checkout (`POST /api/v1/custodian/checkouts/walk-in`) without prior web reservation (`FR-15`) and request rejection (`POST /api/v1/custodian/requests/:id/reject` - `FR-16`).

• **Mapped Requirements:** `FR-13`, `FR-14`, `FR-15`, `FR-16`, `BR-01`, `BR-02`, `BR-03`, `BR-05`, `BR-08`, `NFR-03`.

• **Verification Approach:** Concurrency integration tests simulating duplicate checkouts; role authorization tests (Borrower getting 403); UI E2E test of complete checkout journey.

F-05: Equipment Return & Condition Assessment

• **Scope in Sprint 2:** Custodian selects an active loan, enters return inspection notes and condition grade (e.g., `GOOD`, `MINOR_WEAR`, `DAMAGED`) via `POST /api/v1/custodian/returns`. Loan status set to `RETURNED`; item status reverted to `AVAILABLE`.

• **Mapped Requirements:** `FR-17`, `BR-01`, `BR-07`.

F-06: Due Date Enforcement & Overdue Tracking

• **Scope in Sprint 2:** Background overdue evaluation comparing persisted `dueDate` with `now()` for loans in status `ACTIVE`. Flags delinquent loans as `OVERDUE` and displays visual warning badges on Borrower and Custodian dashboards.

• **Mapped Requirements:** `FR-18`, `BR-05`, `BR-06`.

F-07: Catalog Filtering, Full-Text Search & Category Navigation

• **Scope in Sprint 2:** Multi-parameter search endpoint (`GET /api/v1/items?search=...&category=...&availability=...`). Fast indexed lookup. Responsive UI filter sidebar.

• **Mapped Requirements:** `FR-05`, `FR-06`, `NFR-04`, `NFR-06`.

F-08: Inventory Audit History & Custodial Reporting

• **Scope in Sprint 3:** Append-only transaction log table tracking all state transitions with timestamps and actor IDs (`GET /api/v1/custodian/logs`). Summary dashboard metrics (utilization rate, most borrowed equipment, active loans vs total capacity).

• **Mapped Requirements:** `BR-08`, `NFR-05`.

Source: docs/traceability.md

Requirements Traceability Matrix

CPE Equipment & Hardware Inventory and Borrowing System (EquipFlow)

Document ID: `TRACE-01`
Version: `1.1.0`
Status: Approved **Deliverable:** Project Deliverable 1 — Part B

1. Traceability Matrix Overview

This matrix maps all Functional Requirements (FR), Business Rules (BR), and Non-Functional Requirements (NFR) from the Software Requirements Specification to their delivering Features (Feature Inventory), target development sprints, and planned verification methods.

2. Requirements-to-Feature-to-Sprint Traceability Matrix

Req ID	Req Type	Description Summary	Feature ID	Target Sprint	Planned Verification Method & Test ID
FR-01	Functional	User login & JWT Bearer issuance	**F-01**	Sprint 1	API Integration Test (`TEST-AUTH-01`)
FR-02	Functional	Current user identity (`GET /api/v1/auth/me`)	**F-01**	Sprint 1	API Integration Test (`TEST-AUTH-02`)
FR-03	Functional	Logout & per-token jti revocation	**F-01**	Sprint 1	API Test verifying 401 on revoked token & 200 on concurrent session (`TEST-AUTH-03`)
FR-04	Functional	Server-side role authorization guard	**F-01**	Sprint 1	API Integration Test (`TEST-AUTH-04`)
FR-05	Functional	Browse equipment catalog (`/api/v1/items`)	**F-02**	Sprint 1	UI Journey / E2E (`TEST-CAT-01`)
FR-06	Functional	Inspect equipment details & serial	**F-02**	Sprint 1	API Integration Test (`TEST-CAT-02`)
FR-07	Functional	Custodian create new equipment	**F-02**	Sprint 2	API & UI Test (Sprint 2 contract)
FR-08	Functional	Custodian manage item maintenance	**F-02**	Sprint 2	API & UI Test (Sprint 2 contract)
FR-09	Functional	Borrower submit borrow request	**F-03**	Sprint 1	E2E & API Integration (`TEST-REQ-01`)
FR-10	Functional	Borrower view personal requests (`/requests/my`)	**F-03**	Sprint 1	UI Component & API Test (`TEST-REQ-02`)
FR-11	Functional	Cross-user data isolation (privacy)	**F-03**	Sprint 1	API Security Test (`TEST-REQ-03`)
FR-12	Functional	Borrower cancel pending request	**F-03**	Sprint 2	API & UI Test (Sprint 2 contract)
FR-13	Functional	Custodian pending request triage queue	**F-04**	Sprint 1	UI Journey / E2E (`TEST-CHK-01`)
FR-14	Functional	Custodian checkout reserved item & 7d due date	**F-04**	Sprint 1	E2E & DB Integration (`TEST-CHK-02`)
FR-15	Functional	Walk-in checkout by Custodian	**F-04**	Sprint 2	API & UI Test (Sprint 2 contract)
FR-16	Functional	Custodian reject borrow request	**F-04**	Sprint 2	API & UI Test (Sprint 2 contract)
FR-17	Functional	Process return & condition check	**F-05**	Sprint 2	API & DB Integration (Sprint 2 contract)
FR-18	Functional	Overdue loan calculation & badges	**F-06**	Sprint 2	Unit & API Integration (Sprint 2 contract)
BR-01	Business Rule	Item AVAILABLE to request; RESERVED at checkout	**F-03, F-04**	Sprint 1	Domain Unit & API Test (`TEST-RULE-01`)
BR-02	Business Rule	Exclusive allocation & duplicate conflict rejection	**F-03, F-04**	Sprint 1	Concurrency Stress Test (`TEST-RULE-02`)
BR-03	Business Rule	Role segregation: Borrowers cannot checkout	**F-04**	Sprint 1	API Security Test (`TEST-RULE-03`)
BR-04	Business Rule	Active Quota: Count(requests) + Count(loans) ≤ 3	**F-03**	Sprint 1	Domain Unit & API Test (`TEST-RULE-04`)

Req ID	Req Type	Description Summary	Feature ID	Target Sprint	Planned Verification Method & Test ID
BR-05	Business Rule	7-day default loan period saved on checkout	**F-04, F-06**	Sprint 1/2	DB Assertion (`TEST-CHK-02`) & Sprint 2 Engine
BR-06	Business Rule	48-hour request expiration	**F-06**	Sprint 2	Scheduler Unit Test (Sprint 2 contract)
BR-07	Business Rule	Unidirectional loan state transitions	**F-05**	Sprint 2	State Machine Unit Test (Sprint 2 contract)
BR-08	Business Rule	Custodian accountability & audit actor	**F-04, F-08**	Sprint 1/3	DB Assertion (`TEST-CHK-02`) & Sprint 3 Audit
NFR-01	Non-Funct.	Passwords hashed with bcrypt (≥ 10); local seeds	**F-01**	Sprint 1	Unit & Seed DB Query (`TEST-NFR-01`)
NFR-02	Non-Funct.	401/403 backend route protection & revocation	**F-01**	Sprint 1	Supertest Route Suite (`TEST-NFR-02`)
NFR-03	Non-Funct.	Concurrency conflict safe rejection (HTTP 409)	**F-04**	Sprint 1	Concurrency Test (`TEST-RULE-02`)
NFR-04	Non-Funct.	API query latency < 200ms on 50 items	**F-07**	Sprint 2	Performance Benchmark Test
NFR-05	Non-Funct.	Persistence across server restart & migrations	**F-02, F-04**	Sprint 1	Server Restart Integration (`TEST-NFR-05`)
NFR-06	Non-Funct.	Responsive viewport UI (375px to 1024px)	All UI	Sprint 2	Playwright Responsive Test Suite

3. Sprint Coverage Summary

Sprint	Included Feature IDs	Covered Functional Requirements	Covered Business Rules	Covered Non-Functional Requirements	Status
Sprint 1	`F-01`, `F-02` (read), `F-03`, `F-04` (core)	`FR-01`, `FR-02`, `FR-03`, `FR-04`, `FR-05`, `FR-06`, `FR-09`, `FR-10`, `FR-11`, `FR-13`, `FR-14`	`BR-01`, `BR-02`, `BR-03`, `BR-04`, `BR-05` (save), `BR-08`	`NFR-01`, `NFR-02`, `NFR-03`, `NFR-05`	**Completed & Verified (`v0.1.0-sprint1`)**
Sprint 2	`F-02` (manage), `F-04` (walk-in), `F-05`, `F-06`, `F-07`, `F-03` (cancel), `F-04` (reject)	`FR-07`, `FR-08`, `FR-12`, `FR-15`, `FR-16`, `FR-17`, `FR-18`	`BR-05` (overdue), `BR-06`, `BR-07`	`NFR-04`, `NFR-06`	Planned (Deliverable 2 Target: 29 Nov)
Sprint 3	`F-08`, Integration & Hardening	Cross-feature hardening & analytics	Full system compliance	Full system regression & stability	Planned (Final Delivery Target: 13 Dec)

4. Coverage Gap Analysis & Approved Changes

• **Baseline Completeness:** 100% of the SRS requirements are mapped to features and designated sprints. No orphan requirements exist.

• **Addressed Peer Review Changes:** `BR-01` clarified: Requesting an item requires status `AVAILABLE` and transitions it to `RESERVED`. Checkout requires the item to be `RESERVED` for the matching pending request. `BR-04` clarified: Quota is defined as Count(Pending Requests) + Count(Active Loans) ≤ 3. `FR-03` updated: Server-side token invalidation is tested by asserting HTTP 401 on subsequent requests. `FR-15` (Walk-in checkout) moved to Sprint 2. `BR-05` clarified: 7-day `dueDate` is calculated and saved on checkout in Sprint 1; overdue processing logic added in Sprint 2.

Part C · System-level design

Source: docs/system-sds.md

System-Level Software Design Specification (SDS)

CPE Equipment & Hardware Inventory and Borrowing System (EquipFlow)

Document ID: `SDS-SYS-01`
Version: `1.1.0`
Status: Approved **Deliverable:** Project Deliverable 1 — Part C
Due Date: 15 November 2026 at 23:59 (Aligned with LEB2)

1. System Architecture & Tech Stack

1.1 Architectural Overview

EquipFlow is structured as a **modular full-stack application** composed of three logical tiers:

1. **Client Tier (Presentation):** Single Page Application (SPA) built with React 18, Vite, and TypeScript.

2. **Application / API Tier (Business Logic):** RESTful service built with Node.js, Express, and TypeScript, structured into Controllers, Services, and Domain Validators.

3. **Persistence Tier (Data):** Relational database managed through Prisma ORM (SQLite for local deterministic development and testing; PostgreSQL in a Docker container for production / GCP deployment).

Browser	Application/API	Persistence
React + Vite client	Express routes, authentication and role guards	Prisma Client
HTTP JSON with bearer token →	Domain services and transactions →	SQLite locally; PostgreSQL plan for Lab 5

1.2 Technology Stack Rationale

Layer	Selected Technology	Version	Rationale & Justification
Frontend Framework	**React** with **Vite**	`18.3.x` / `6.x`	High responsiveness, fast HMR during development, strong component reusability, and continuity with TokTickIT labs 1–4.
Language	**TypeScript**	`5.7.x`	End-to-end type safety between client API calls, backend routes, and database models, eliminating common runtime type errors.
Styling & UI	**Bootstrap 5**	`5.3.x`	Clean, standardized utility classes, responsive grid system (mobile/tablet/desktop), and zero styling overhead.
Backend Runtime	**Node.js + Express**	`22.x` / `4.21.x`	Lightweight, fast event-driven HTTP request handling with robust middleware support for auth and role checks.
ORM / Data Access	**Prisma ORM**	`5.22.x`	Type-safe queries, automated database migrations, declarative schema definition, and seamless seed scripting.
Database Engine	**SQLite (Dev/Test)** **PostgreSQL (GCP)**	SQLite 3 PostgreSQL 16	SQLite provides zero-dependency, isolated, deterministic local testing. PostgreSQL on GCP Cloud SQL provides production durability for Lab 5.
Testing Tools	**Vitest**, **Supertest**, **Playwright**	Vitest 2.x Supertest 7.x Playwright 1.63.x	Vitest for lightning-fast unit and domain rule testing; Supertest for API and auth integration; Playwright for browser E2E journeys.

1.3 Communication & Business Rule Enforcement Boundary

• **Strict Backend Enforcement:** All domain invariants and business rules (`BR-01` to `BR-08`) are enforced exclusively within the **Backend Service Layer** (`src/services/`).

• The client UI disables buttons and provides client-side validation solely for user convenience. A hidden button or disabled input is never treated as authorization.

• **Transactional Concurrency Protection:** Every state-mutating operation (request creation, checkout, return) executes inside a database transaction (`prisma.$transaction`) to guarantee atomicity and prevent race conditions.

2. Domain Data Model & Entity Specifications

2.1 Entity Relationship Diagram (ERD)

Entity	Relationships and constraints
User	Submits borrow requests, holds loans as borrower or custodian, performs audit actions
EquipmentItem	Can appear in multiple requests and loans and can be referenced by audit records
BorrowRequest	Belongs to a borrower and an item; can result in zero or one loan
Loan	References equipment, borrower, and checkout custodian; may link to one request
AuditLog	Records an actor and optionally an equipment item; reporting planned for Sprint 3

2.2 Entity Field Dictionary & Integrity Constraints

1. `User` Entity

- `id` (String, UUID/CUID, Primary Key)
- `email` (String, Unique, Not Null) — Institutional email address used for login.
- `passwordHash` (String, Not Null) — bcrypt hash (salt rounds ≥ 10). Plaintext never stored.
- `fullName` (String, Not Null) — User display name.
- `studentOrStaffId` (String, Unique, Not Null) — Academic identifier (e.g., student ID `67070501026`).
- `role` (Enum: `BORROWER`, `CUSTODIAN`, Not Null) — User role for access control.
- `isActive` (Boolean, Default `true`) — Disabled users cannot log in.

1.1 `RevokedToken` Entity

- `jti` (String, Primary Key) — Unique JWT identifier string.
- `expiresAt` (DateTime, Not Null) — Token expiry date/time. Records past this timestamp can be purged.
- `createdAt` (DateTime, Default `now()`) — Audit timestamp of when logout occurred.

2. `EquipmentItem` Entity

- `id` (String, UUID/CUID, Primary Key)
- `assetTag` (String, Unique, Not Null) — Scannable department inventory barcode/tag (e.g., `CPE-OSC-001`).
- `name` (String, Not Null) — Equipment title (e.g., "Rigol DS1054Z Digital Oscilloscope").
- `category` (String, Not Null) — Classification (e.g., "Measurement", "Microcontrollers", "FPGA").
- `modelNumber` (String, Nullable) — Manufacturer part number.
- `serialNumber` (String, Unique, Not Null) — Manufacturer hardware serial number.
- `condition` (Enum: `NEW`, `GOOD`, `FAIR`, `DAMAGED`, Default `GOOD`).
- `status` (Enum: `AVAILABLE`, `RESERVED`, `CHECKED_OUT`, `MAINTENANCE`, `RETIRED`, Default `AVAILABLE`).

3. `BorrowRequest` Entity

- `id` (String, UUID/CUID, Primary Key)
- `borrowerId` (String, Foreign Key → `User.id`, Not Null)
- `itemId` (String, Foreign Key → `EquipmentItem.id`, Not Null)
- `status` (Enum: `PENDING`, `APPROVED`, `REJECTED`, `CANCELLED`, `EXPIRED`, Default `PENDING`).
- `requestNote` (String, Nullable) — Stated academic purpose of request.

• `rejectionReason` (String, Nullable) — Recorded when rejected by custodian.

4. `Loan` **Entity**

• `id` (String, UUID/CUID, Primary Key)

• `requestId` (String, Foreign Key → `BorrowRequest.id`, Unique, Nullable)

• `itemId` (String, Foreign Key → `EquipmentItem.id`, Not Null)

• `borrowerId` (String, Foreign Key → `User.id`, Not Null)

• `custodianId` (String, Foreign Key → `User.id`, Not Null) — Accountable staff recording checkout (`BR-08`).

• `status` (Enum: `ACTIVE`, `RETURNED`, `OVERDUE`, Default `ACTIVE`).

• `checkedOutAt` (DateTime, Default `now()`).

• `dueDate` (DateTime, Not Null) — Authoritative deadline (`checkedOutAt + 7 days`, `BR-05`).

• `returnedAt` (DateTime, Nullable).

• `returnCondition` (Enum, Nullable).

3. UI, API & Access Conventions

3.1 Screen Hierarchy & Navigation

The UI provides distinct views based on authenticated role identity:

```
[ Public / Unauthenticated ]
    ├── Login View (/login)

[ Authenticated: Borrower ]
    ├── Catalog Browser View (/catalog)
    │      └── Item Detail Modal (Specs, Status, "Request Item" button)
    └── My Requests & Loans View (/my-requests)
           ├── Active Loans List (Item details, Checked out date, Due date)
           └── Pending Requests List (Status badges: PENDING, APPROVED)

[ Authenticated: Custodian ]
    ├── Custodian Operations Desk (/custodian/desk)
    │      ├── Pending Requests Queue ("Approve & Checkout", "Reject" actions)
    │      └── Active Loans Queue ("Process Return" action)
    ├── Inventory Catalog Manager (/custodian/inventory) [Sprint 2]
    └── System Audit Logs View (/custodian/logs) [Sprint 3]
```

3.2 Shared UI Behaviors

• **Loading States:** All asynchronous fetch actions render a standardized loading spinner or skeleton card.

• **Empty States:** When a list is empty (e.g., zero pending requests), a contextual empty state card is rendered (`"No pending requests found."`).

• **Validation Feedback:** Forms display red inline helper text upon field validation failure and disable submit buttons while the request is in flight to prevent double submissions.

• **Alerts & Toasts:** Action confirmations (e.g., `"Checkout recorded successfully"`) appear as floating dismissible alerts.

3.3 Authoritative API Conventions & Error Structure

All API endpoints follow RESTful URI conventions under `/api/v1/` and return a standard JSON envelope:

Successful Response Format

Error Response Format

HTTP Status Codes

• `200 OK`: Request succeeded.

- `201 Created`: Resource successfully created.
- `400 Bad Request`: Validation failure or business rule violation (`BR-01`, `BR-04`).
- `401 Unauthorized`: Missing, invalid, or revoked authentication token.
- `403 Forbidden`: Authenticated user lacks required role or is attempting cross-user access (`BR-03`, `FR-11`).
- `404 Not Found`: Target resource does not exist.
- `409 Conflict`: Concurrency conflict (e.g., duplicate checkout attempt `BR-02`, concurrent quota overflow).
- `500 Internal Server Error`: Unhandled server exception (sanitized in production).

3.4 Role & Permission Matrix

All endpoints strictly use the `/api/v1/` prefix:

Operation / Endpoint	Anonymous	Borrower	Custodian	Ownership & Business Rule Guard	Sprint
`POST /api/v1/auth/login`	**Allowed**	Allowed	Allowed	Public authentication	Sprint 1
`POST /api/v1/auth/logout`	Denied (401)	**Allowed**	**Allowed**	Server records revocation; token rendered invalid	Sprint 1
`GET /api/v1/auth/me`	Denied (401)	**Allowed**	**Allowed**	Returns authenticated profile and role	Sprint 1
`GET /api/v1/items`	Denied (401)	**Allowed**	**Allowed**	View all catalog items and statuses	Sprint 1
`GET /api/v1/items/:id`	Denied (401)	**Allowed**	**Allowed**	View single item details and specifications	Sprint 1
`POST /api/v1/requests`	Denied (401)	**Allowed**	Denied (403)	Item must be `AVAILABLE` (`BR-01`); Active Quota ≤ 3 (`BR-04`)	Sprint 1
`GET /api/v1/requests/my`	Denied (401)	**Allowed**	Denied (403)	Filtered strictly by authenticated `userId` (`FR-11`)	Sprint 1
`POST /api/v1/requests/:id/cancel`	Denied (401)	**Allowed**	Denied (403)	Can only cancel owned pending requests	Sprint 2
`GET /api/v1/custodian/requests`	Denied (401)	Denied (403)	**Allowed**	Lists all pending department requests	Sprint 1
`POST /api/v1/custodian/checkouts`	Denied (401)	Denied (403)	**Allowed**	Item must be `RESERVED` for matching request (`BR-01`, `BR-02`)	Sprint 1
`POST /api/v1/custodian/checkouts/walk-in`	Denied (401)	Denied (403)	**Allowed**	Item must be `AVAILABLE` for walk-in (`FR-15`)	Sprint 2
`POST /api/v1/custodian/requests/:id/reject`	Denied (401)	Denied (403)	**Allowed**	Custodian rejects with reason; item → `AVAILABLE`	Sprint 2
`POST /api/v1/custodian/returns`	Denied (401)	Denied (403)	**Allowed**	Custodian records return condition; item → `AVAILABLE`	Sprint 2
`POST /api/v1/custodian/items`	Denied (401)	Denied (403)	**Allowed**	Custodian registers new equipment asset	Sprint 2
`PATCH /api/v1/custodian/items/:id/status`	Denied (401)	Denied (403)	**Allowed**	Custodian marks item `MAINTENANCE` or `RETIRED`	Sprint 2

3.5 Single JWT Transport & Per-Token Revocation Contract

- **Single Transport:** All authenticated requests must pass the JWT token in the HTTP Authorization header: `Authorization: Bearer <jwt-token>`

• **Payload Structure:** `{ "userId": "usr_cuid123", "role": "BORROWER", "email": "borrower@cpe.kmutt.ac.th", "jti": "550e8400-e29b-41d4-a716-446655440000", "iat": 1731448800, "exp": 1731535200 }`

• **Authoritative Logout Contract & Enforcement across Instances (`FR-03`): Logout Action:** When a user logs out, the client issues `POST /api/v1/auth/logout` presenting its Bearer token. **Database Persistence:** The backend decodes the token, extracts `jti` and `exp`, and creates a record in the database: `await prisma.revokedToken.create({ data: { jti: decoded.jti, expiresAt: new Date(decoded.exp * 1000) } });` **Multi-Instance Enforcement:** Because `RevokedToken` is stored in the shared relational database (SQLite locally, Cloud SQL in production), every backend instance or container synchronously observes the revocation. The authentication middleware (`requireAuth`) queries: `const isRevoked = await prisma.revokedToken.findUnique({ where: { jti: decoded.jti } }); if (isRevoked) { return res.status(401).json({ success: false, error: { code: "TOKEN_REVOKED", message: "This session has been logged out." } }); }` **Multi-Session Independence:** Because revocation is keyed strictly by the unique `jti` of that specific token, logging out on Device A does not invalidate concurrent active sessions on Device B. **Verification Specification (`TEST-AUTH-03`):** Step 1: Login user as Session 1 → obtain `token1` (with `jti_1`). Step 2: Login same user as Session 2 → obtain `token2` (with `jti_2`). Step 3: Call `GET /api/v1/auth/me` with `token1` → assert HTTP 200. Step 4: Call `GET /api/v1/auth/me` with `token2` → assert HTTP 200. Step 5: Call `POST /api/v1/auth/logout` with `token1` → assert HTTP 200. Step 6: Call `GET /api/v1/auth/me` with `token1` → assert HTTP 401 (`TOKEN_REVOKED`). Step 7: Call `GET /api/v1/auth/me` with `token2` → assert HTTP 200 (proving concurrent session remains valid).

4. Database Setup, Migrations & Deployment Path

4.1 Reproducible Database Strategy across Environments

Prisma migration files are SQL-dialect specific. To ensure 100% reproducibility across Local Development, CI, and Cloud deployment without provider incompatibility:

A. Local Development & Automated Testing (Sprint 1 Baseline)

• **Provider:** SQLite (`datasource db { provider = "sqlite" }` in `prisma/schema.prisma`).

• **Environment Variable:** `DATABASE_URL="file:./dev.db"` (for dev) and `DATABASE_URL="file:./test.db"` (for isolated automated testing).

• **Migration Directory:** `prisma/migrations/` (committed to Git).

• **Migration Generation & Application:** `# Generate and apply local migration npx prisma migrate dev --name init`

• **Deterministic Seed Command:** `# Execute reproducible seed data npx prisma db seed` *Seed Data Details:* Creates local test accounts `borrower@cpe.kmutt.ac.th` and `custodian@cpe.kmutt.ac.th` with password `[local seed password omitted]` (strictly restricted to local dev), plus 6 seeded equipment items across statuses.

B. Continuous Integration (CI Pipeline in Sprint 1)

• Runs inside GitHub Actions on PRs into `sprint-1-staging` and `main`.

• **Migration Validation:** CI explicitly validates committed migrations against a fresh SQLite database rather than using `db push`: `# Drops DB and applies all committed migrations from scratch to guarantee reproducibility npx prisma migrate reset --force --skip-seed npx prisma db seed npm test`

• Ensures that committed migrations execute cleanly from scratch with zero drift before PR approval.

C. Cloud Deployment & GCP Cloud Run (Lab 5 Release)

• **Provider & Dedicated Schema Directory:** PostgreSQL (`datasource db { provider = "postgresql" }`). To prevent Prisma migration collision with the SQLite baseline, PostgreSQL configuration resides in its own isolated schema directory: Schema path: `prisma/postgres/schema.prisma` Migration directory: `prisma/postgres/migrations/` (located alongside `schema.prisma` as required by Prisma engine)

• **CI & Pre-Deployment PostgreSQL Verification:** Prior to GCP deployment, the CI pipeline boots an ephemeral PostgreSQL service container (`services: postgres:16`). Runs client generation and pre-deploy migration check against a clean database: `# Generate Prisma Client from PostgreSQL schema npx prisma generate --schema=prisma/postgres/schema.prisma # Apply PostgreSQL migrations to clean test database npx prisma migrate deploy --schema=prisma/postgres/schema.prisma # Seed PostgreSQL test database npm run seed:pg # Execute test suite against PostgreSQL npm test` Proves that the PostgreSQL migrations and seed scripts succeed on a clean, empty database before deployment.

• **GCP Cloud Run Deployment Command:** Uses Cloud SQL Auth Proxy connecting to GCP Cloud SQL (PostgreSQL 16): `DATABASE_URL="postgresql://equipflow_user:${DB_PASSWORD}@127.0.0.1:5432/equipflow?sslmode=disable" \ npx prisma migrate deploy --schema=prisma/postgres/schema.prisma`

• **Credential Segregation:** No sample accounts or local passwords are used in production. Production credentials (`DATABASE_URL`, `JWT_SECRET`) are provisioned via **GCP Secret Manager** and injected into the Cloud Run container at runtime. Initial administrative cloud account is provisioned via a dedicated onboarding script, not via development seeds.

4.2 Multi-Stage Docker Packaging Readiness & Release Boundary

Distinction between Sprint 1 Release and Lab 5 Deployment

1. **Sprint 1 Accepted Release (`v0.1.0-sprint1`): Scope:** Complete, locally runnable vertical slice on SQLite with passing Vitest, Supertest, and Playwright tests, comprehensive setup documentation in `README.md`, and seed scripts. **Staging & Release Flow:** Features merge into `sprint-1-staging`, followed by a reviewed release PR merged into `main` and tagged as `v0.1.0-sprint1`. **Exclusions:** Does *not* require Docker containers, live GCP URLs, or active cloud pipelines (strictly per Deliverable 1 handout).

2. **Lab 5 Deployment Release (`v0.2.0-lab5`): Branch & Staging Flow:** Branches off `sprint-1-staging` (incorporating `v0.1.0-sprint1`) onto feature branch `feat/lab5-cloud-deployment`. Adds the multi-stage `Dockerfile`, `.github/workflows/ci-cd.yml`, `prisma/postgres/schema.prisma`, and Cloud Run deployment pipeline. **PR 1 (Feature into Staging):** `feat/lab5-cloud-deployment` → `sprint-1-staging`, reviewed and approved by teammate. **PR 2 (Release into Main):** Reviewed release PR from `sprint-1-staging` → `main`. **Deployment & Tag:** The merge commit on `main` is tagged as `v0.2.0-lab5` and deployed directly to GCP Cloud Run.

Docker Multi-Stage Build Architecture (for Lab 5)

• **Stage 1 (Build Client):** Vite builds optimized static assets into `client/dist/`.

• **Stage 2 (Build Server):** TypeScript compiler compiles backend into `server/dist/`.

• **Stage 3 (Runtime):** Lightweight Node.js 22 Alpine runtime containing compiled backend, serving static client files, listening on `PORT` (default 8080 for Cloud Run).

Part D · Three-sprint plan

Source: docs/sprints/plan.md

Three-Sprint Development Plan

CPE Equipment & Hardware Inventory and Borrowing System (EquipFlow)

Document ID: `PLAN-01`
Version: `1.1.0`
Status: Approved **Deliverable:** Project Deliverable 1 — Part D
Due Date: 15 November 2026 at 23:59 (Aligned with LEB2 Assignment)

1. Executive Summary & Delivery Milestones

The EquipFlow engineering plan spans three structured iterations across the course semester. The project follows a progressive vertical-slice methodology: establishing an end-to-end authenticated domain baseline in Sprint 1, deploying it via Lab 5, expanding core lifecycle operations in Sprint 2, and hardening the product for final release and verification in Sprint 3.

Checkpoint	Feature milestones
Sprint 1 · 15 Nov	F-01 authentication; F-02 catalog/seed; F-03 borrowing; F-04 checkout; Deliverable 1
Lab 5 · 15 Nov	Docker and Cloud Run deployment track
Sprint 2 · 29 Nov	F-05 returns; F-06 overdue; FR-15 walk-in; catalog management and search
Sprint 3 · 13 Dec	F-08 auditing/reporting; regression; packaging and video

2. Sprint Breakdown

Sprint 1: Working Domain Slice & Baseline

• **Milestone Deadline:** Deliverable 1 (15 November 2026 at 23:59 | 80 Points)

• **Sprint Goal:** Establish the engineering baseline and release a small, fully functional vertical slice demonstrating secure login, role separation, equipment browsing, item requesting, and custodian checkout with strict conflict rejection, 7-day due-date calculation, and real database persistence.

• **Included Features:** `F-01`: Authentication, Session, and Role Identity (`FR-01`, `FR-02`, `FR-03`, `FR-04`, `NFR-01`, `NFR-02`). `F-02` **(Core)**: Serialized Equipment Catalog view & Seed data (`FR-05`, `FR-06`, `BR-01`, `NFR-05`). `F-03` **(Core)**: Borrower Request Submission & Personal History (`FR-09`, `FR-10`, `FR-11`, `BR-01`, `BR-02`, `BR-04`). `F-04` **(Core)**: Custodian Counter Checkout & Conflict Prevention (`FR-13`, `FR-14`, `BR-01`, `BR-02`, `BR-03`, `BR-05` [record due date], `BR-08`, `NFR-03`).

• **Due Date Handling in Sprint 1:** Upon checkout, the backend automatically calculates and stores `dueDate = checkedOutAt + 7 days` on the `Loan` record.

• **Dependencies:** None (initial development sprint).

• **Expected Completion Outcome:** A working increment merged to `main` with reproducible SQLite seed scripts. Borrower can log in, request an available item (status becomes `RESERVED`), and see their pending request in personal history. Custodian can log in, view pending requests, and checkout the reserved item, moving it to `CHECKED_OUT` and saving the 7-day due date. Rejection rules verified: cannot request an already reserved/checked-out item; cannot exceed combined quota of 3; concurrent checkouts for the same asset are rejected with HTTP 409 Conflict. Role restriction verified: Borrower attempting custodian checkout receives HTTP 403 Forbidden. Logout verified: Token invalidated on server side; subsequent calls return HTTP 401. Automated Vitest, Supertest, and Playwright suites passing.

Parallel Track: Lab 5 (CI/CD, Docker & GCP Deployment)

• **Milestone Deadline:** Lab 5 (15 November 2026 at 23:59 | 100 Lab Points)

• **Objective:** Containerize the accepted Sprint 1 release on `main`, configure PostgreSQL migrations, establish a GitHub Actions CI quality gate, and deploy to GCP Cloud Run.

• **Release & Staging Pipeline Boundary: Accepted Sprint 1 Release (`v0.1.0-sprint1`):** The release candidate merged into `main` from `sprint-1-staging` for Deliverable 1 contains the local SQLite baseline, seed scripts, passing automated tests, and local documentation. It does not include active GCP deployment or cloud pipelines. **Lab 5 Staging & Deployment Flow (`v0.2.0-lab5`):** Lab 5 development branches off `sprint-1-staging` (incorporating `v0.1.0-sprint1`) onto feature branch `feat/lab5-cloud-deployment`. Introduces multi-stage `Dockerfile`, `.github/workflows/ci-cd.yml`, dedicated PostgreSQL schema `prisma/postgres/schema.prisma` with `prisma/postgres/migrations/`, and GCP Cloud Run deployment configuration. **Feature PR into Staging:** PR from `feat/lab5-cloud-deployment` → `sprint-1-staging`. Teammate reviews and approves with substantive comments. **Staging Integration & CI Checks:** CI verifies Prisma client generation (`npx prisma generate --schema=prisma/postgres/schema.prisma`), applies PostgreSQL migrations to ephemeral PostgreSQL container (`npx prisma migrate deploy --schema=prisma/postgres/schema.prisma`), seeds, and executes test suites. **Reviewed Release PR into Main:** Reviewed release PR opened from `sprint-1-staging` → `main`. **Deployment & Tagging:** Merged into `main` and tagged as `v0.2.0-lab5`. The live GCP Cloud Run instance hosts the exact commit associated with tag `v0.2.0-lab5` deployed directly from `main`.

Sprint 2: Core Lifecycle Expansion & Asset Management

• **Milestone Deadline:** Deliverable 2 (29 November 2026 at 23:59 | 70 Points)

• **Sprint Goal:** Complete the circular borrowing lifecycle by introducing return processing with condition inspections, active overdue background processing, direct walk-in checkouts, custodian inventory editing, and multi-criteria catalog search.

• **Included Features:** `F-05`: Equipment Return & Condition Assessment (`FR-17`, `BR-01`, `BR-07`). `F-06`: Due Date Enforcement & Overdue Tracking (`FR-18`, `BR-05`, `BR-06` - evaluates persisted `dueDate` against `now()` and flags delinquent loans). `F-04` **(Expansion)**: Direct Walk-in Checkout without prior reservation (`FR-15`), and Request Rejection with reasons (`FR-16`). `F-02` **(Expansion)**: Custodian Catalog Administration (`FR-07`, `FR-08`). `F-07`: Catalog Filtering, Search & Category Navigation (`FR-05`, `FR-06`, `NFR-04`, `NFR-06`). `F-03` **(Expansion)**: Borrower Request Cancellation (`FR-12`).

• **Dependencies:** Completed Sprint 1 database entities, deployed Lab 5 infrastructure, and authenticated endpoints.

• **Expected Completion Outcome:** Complete lifecycle from Request → Checkout → Active Loan → Return → Available. Overdue loan indicator flags visually and logically active on dashboards. Walk-in checkouts functional for immediate counter service.

Sprint 3: Completion, Auditing, Hardening & Final Release

• **Milestone Deadline:** Final Project Delivery (13 December 2026 at 23:59 | 100 Points)

• **Sprint Goal:** Finalize the complete product with append-only audit trail logging, custodial utilization metrics, defect resolution from Lab 6 UAT, full automated regression coverage, and final documentation/demonstration video.

• **Included Features:** `F-08`: Inventory Audit History & Custodial Reporting (`BR-08`, `NFR-05`). **System Hardening:** Error boundaries, edge-case validation, UI accessibility (WCAG 2.1 AA), and responsive polish. **Full Regression:** Comprehensive end-to-end regression tests across all features. **Release Packaging:** Immutable production release tag, final report PDF, and 10-minute demonstration video.

• **Dependencies:** Completed Sprint 1 & 2 features, Lab 6 UAT feedback, and Lab 7 maintenance change.

3. Team Work Allocation & Responsibilities

Student Role / Focus	Primary Sprint 1 Ownership	Primary Sprint 2 Ownership	Primary Sprint 3 Ownership
Partner 1 (Lead Spec / Backend)	• Authentication & JWT Bearer middleware (`F-01`) • Custodian Checkout Service & Concurrency (`F-04`) • Supertest integration test suite	• Return & condition workflow (`F-05`) • Overdue calculation engine (`F-06`) • Walk-in checkout endpoint (`FR-15`) • Lab 5 Docker containerization	• Audit logging service (`F-08`) • Performance testing & regression suite • Final deployment verification
Partner 2 (Lead UI / Flow)	• Catalog view & seed setup (`F-02`) • Borrower request submission UI (`F-03`) • Playwright E2E test suite	• Catalog admin & search UI (`F-07`, `F-02`) • Custodian return desk UI (`F-05`) • Responsive styling & UX review	• Utilization reporting dashboard (`F-08`) • Cross-team UAT remediation (Lab 6) • Final demonstration video recording

4. Scope Feasibility & Risk Mitigation

Identified Risk	Impact	Probability	Mitigation Strategy
Concurrency deadlocks on physical asset checkouts	High	Low	Use Prisma atomic transactions (`$transaction`) with optimistic locking on item status (`RESERVED` / `AVAILABLE` guard). Second attempt returns HTTP 409 Conflict.
Scope creep (e.g. barcode scanner hardware, fine payments)	High	Medium	Explicitly bound baseline in SRS Section 3.2. Fines and physical scanners are strictly excluded from baseline.
Database migration issues between SQLite & PostgreSQL	Medium	Low	Use SQLite for local development/testing with `prisma migrate dev` and PostgreSQL for GCP with `prisma migrate deploy`, keeping dialect-specific migrations organized.
Delay in teammate PR reviews	Medium	Medium	Agree on scheduled daily review windows and pair-programming sessions during sprint workshops.

Part E · Sprint 1 engineering contracts

Source: docs/sprints/sprint-1/specification.md

Sprint 1 Engineering Specification

EquipFlow Hardware Inventory & Borrowing Vertical Slice

Document ID: `SPEC-SPRINT-01` **Version:** `1.0.0` **Status:** Approved (Verified `v0.1.0-sprint1`) **Target Release:** `v0.1.0-sprint1` **Deadline:** 15 November 2026 at 23:59 (LEB2 Assignment Checkpoint)

1. Sprint Goal

Deliver the complete, working, and verified **Sprint 1 vertical slice** for the Department of Computer Engineering Equipment & Hardware Inventory and Borrowing System (**EquipFlow**). The release must establish the engineering baseline across all three tiers (React/Vite client, Node.js/Express backend, and Prisma/SQLite database), demonstrating secure authentication, role segregation (`BORROWER` vs `CUSTODIAN`), serialized equipment catalog browsing, student borrow request submission, and custodian counter checkout with atomic concurrency conflict protection, 7-day due-date calculation, and reproducible seed data.

2. Stakeholder Request Interpretation

The Department of Computer Engineering hardware laboratories maintain high-value testing instruments, microcontroller boards, FPGA kits, and development tools used across undergraduate courses and senior design projects. Currently, custody tracking relies on manual spreadsheets or physical sign-out binders, resulting in double-allocations, missing equipment, and no real-time visibility for students.

Stakeholders require a modern, dependable, and role-governed platform:

1. **Students (Borrowers)** need a self-service portal to browse real-time hardware availability, inspect specifications, and submit requests without visiting the counter blindly. Furthermore, student borrowing must be bounded by a fair-usage quota (≤ 3 active items) and protected by strict privacy boundaries so no student can view or tamper with another student's loan records.

2. **Lab Technicians (Custodians)** require an authoritative, tamper-resistant triage queue to inspect incoming requests and record the physical handover (checkout). The system must guarantee that an item cannot be checked out to multiple borrowers simultaneously, and that once checked out, the physical asset status is locked and a 7-day return deadline is recorded.

3. **Department Staff** require strict identity enforcement: users must authenticate securely with hashed passwords, and permissions must be verified on the backend—preventing borrowers from bypassing custodial approval or checking out equipment directly.

3. Scope

3.1 Included Scope

• **Authentication, Session & Access Control**: Seeded user accounts for Borrowers and Custodians. Credential validation via `POST /api/v1/auth/login` with bcrypt hashing (≥ 10 salt rounds). Issuance of signed JWT Bearer tokens containing `{ userId, role, email, jti, exp }`. Authenticated identity endpoint `GET /api/v1/auth/me`. Per-token server-side revocation on `POST /api/v1/auth/logout` persisting token `jti` into `RevokedToken` table while keeping other concurrent sessions active. Server-side route middleware returning HTTP 401 (unauthenticated/revoked) and HTTP 403 (unauthorized role).

• **Serialized Equipment Catalog**: Catalog browsing endpoint `GET /api/v1/items` with real-time status badges (`AVAILABLE`, `RESERVED`, `CHECKED_OUT`, `MAINTENANCE`). Detail view endpoint `GET /api/v1/items/:id` showing manufacturer serial number, category, model, and physical condition. Idempotent local seed script (`prisma/seed.ts`) creating at least 6 distinct hardware units.

• **Borrower Request Submission & Personal History**: Request creation endpoint `POST /api/v1/requests` creating a `PENDING` request and transitioning item status to `RESERVED`. Combined Active Quota enforcement: Count(Pending Requests) + Count(Active Loans) ≤ 3. Personal history endpoint `GET /api/v1/requests/my` returning owned requests and active loans. Strict cross-user data isolation: Borrower A cannot view or access Borrower B's requests.

• **Custodian Triage & Exclusive Checkout**: Custodian queue endpoint `GET /api/v1/custodian/requests` listing all pending requests. Checkout execution endpoint `POST /api/v1/custodian/checkouts` validating that the item is `RESERVED` for the matching request, transitioning item to `CHECKED_OUT`, creating an active `Loan`, and persisting calculated 7-day `dueDate`. Atomic transactional execution (`prisma.$transaction`) guaranteeing conflict rejection (HTTP 409) on duplicate or concurrent checkout attempts. Role restriction: Borrowers forbidden from calling checkout endpoints (HTTP 403).

• **Testing & Quality Baseline**: Unit tests for quota calculation and password hashing. API integration tests via Supertest covering all endpoints, auth guards, and race conditions. End-to-end browser tests via Playwright for complete domain journeys.

3.2 Explicitly Excluded Scope

• Monetary late fines and payment gateway processing.

• Physical barcode/RFID hardware scanner integration (data typed or emulated via keyboard).

• Public self-registration and automated email password reset loops (seeded accounts used).

• Direct walk-in checkouts without prior request (allocated to Sprint 2 - `FR-15`).

• Request cancellation by borrower and request rejection by custodian (allocated to Sprint 2).

• Equipment return processing and condition inspection (allocated to Sprint 2 - `F-05`).

• Overdue background detection engine and warning badges (allocated to Sprint 2 - `F-06`).

• Custodian catalog asset creation/editing (allocated to Sprint 2 - `F-02`).

• Multi-criteria catalog search and filtering (allocated to Sprint 2 - `F-07`).

• Historical audit log reporting and export (allocated to Sprint 3 - `F-08`).

• Docker packaging, GCP Cloud Run deployment, and PostgreSQL cloud migrations (allocated to Lab 5).

4. Functional Requirements (FR)

Authentication & Access Control

• **FR-01**: The system shall authenticate registered users using email and password, returning an authenticated JWT Bearer token upon credential validation.

• **FR-02**: The system shall provide endpoint `GET /api/v1/auth/me` returning the profile and role (`BORROWER` or `CUSTODIAN`) of the currently authenticated user.

• **FR-03**: The system shall provide endpoint `POST /api/v1/auth/logout`. Upon invocation, the backend shall persist the Bearer token's unique identifier (`jti`) into the `RevokedToken` table, causing subsequent requests with that token to be rejected with HTTP 401 Unauthorized while leaving other concurrent sessions valid.

• **FR-04**: The system shall enforce server-side access control on all protected endpoints, denying unauthorized role actions with HTTP 403 Forbidden.

Equipment Catalog

• **FR-05**: The system shall allow Borrowers and Custodians to browse the catalog of equipment items via `GET /api/v1/items`, displaying asset tag, name, category, and real-time availability status.

• **FR-06**: The system shall allow users to inspect detailed equipment specifications, model number, serial number, and physical condition via `GET /api/v1/items/:id`.

Borrower Workflow & Ownership

• **FR-09**: The system shall allow an authenticated Borrower to submit a borrow request for any available equipment item via `POST /api/v1/requests`, transitioning the item to `RESERVED` and creating a `PENDING` request.

• **FR-10**: The system shall allow a Borrower to view all personal borrow requests and active loans via `GET /api/v1/requests/my`.

• **FR-11**: The system shall enforce data ownership isolation, restricting Borrowers to viewing only their own requests/loans and forbidding access to other borrowers' records.

Custodian Triage & Checkout

• **FR-13**: The system shall allow Custodians to view all pending borrow requests across all borrowers via `GET /api/v1/custodian/requests`.

• **FR-14**: The system shall allow a Custodian to record the physical checkout of an item against a matching pending request via `POST /api/v1/custodian/checkouts`, transitioning the item to `CHECKED_OUT`, creating an active `Loan`, and automatically calculating a 7-day `dueDate`.

5. Business Rules (BR)

Inventory & Reservation Rules

• **BR-01: Availability & Reservation Consistency**: An equipment item may only be requested if its status is strictly `AVAILABLE`. Submitting a request transitions the item status immediately from `AVAILABLE` to `RESERVED`. An item in status `CHECKED_OUT`, `MAINTENANCE`, `RETIRED`, or `RESERVED` cannot be requested. Any attempt returns HTTP 400 Bad Request (`ITEM_NOT_AVAILABLE`).

• **BR-02: Exclusive Allocation & Concurrency Lock**: A physical serialized equipment asset can have at most one active reservation or loan at any time. If concurrent checkout requests or competing requests arrive for the same asset, they must be resolved atomically within a database transaction. Exactly one operation succeeds; subsequent or competing attempts must be rejected with HTTP 409 Conflict (`CONCURRENT_CHECKOUT_CONFLICT`).

• **BR-03: Role Segregation on Checkout**: Only authenticated users with the `CUSTODIAN` role are permitted to record checkouts and transition items to `CHECKED_OUT`. Borrowers attempting checkout operations must be rejected with HTTP 403 Forbidden.

• **BR-04: Combined Active Quota per Borrower**: A Borrower's active quota is defined as:
Active Quota = Count(Pending Requests) + Count(Active Loans) The maximum permitted active quota per borrower is three (Active Quota ≤ 3). Any request submission that would cause Active Quota > 3 must be rejected with HTTP 400 Bad Request (`EXCEEDS_ACTIVE_QUOTA`). Concurrent request submissions from the same borrower are evaluated in a serialized transaction; any request that causes the count to exceed 3 is aborted and rejected with HTTP 400.

• **BR-05: Default Loan Duration & Due Date**: Every newly created loan record automatically calculates and persists:
dueDate = checkedOutAt + 7 days The calculated due date is immutable during Sprint 1.

• **BR-08: Custodian Accountability**: Every checkout record must automatically record the authenticated custodian's user ID as `custodianId`. Client payloads cannot override or omit this attribution.

Concurrency Invariant & Evaluation Order

For state mutations (`POST /api/v1/requests`, `POST /api/v1/custodian/checkouts`), the backend enforces the following strict evaluation hierarchy:

1. `401 Unauthorized`: Missing, invalid, or revoked Bearer token.

2. `403 Forbidden`: Role restriction (e.g. Borrower attempting custodian checkout).

3. `400 Bad Request`: Payload validation failure or business quota breach (`EXCEEDS_ACTIVE_QUOTA`).

4. `409 Conflict`: Concurrency conflict or duplicate checkout attempt on an already transitioned asset (`CONCURRENT_CHECKOUT_CONFLICT`).

6. Domain Data Model & Persistence

6.1 Database Schema (Prisma ORM)

The Sprint 1 schema is defined in `prisma/schema.prisma` using the SQLite provider:

```
datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

model User {
  id               String          @id @default(cuid())
  email            String          @unique
  passwordHash     String
  fullName         String
  studentOrStaffId String          @unique
  role             Role            @default(BORROWER)
  isActive         Boolean         @default(true)
  createdAt        DateTime        @default(now())
  updatedAt        DateTime        @updatedAt

  requests         BorrowRequest[]
  loansAsBorrower  Loan[]          @relation("BorrowerLoans")
  loansAsCustodian Loan[]          @relation("CustodianLoans")
  auditLogs        AuditLog[]
}

enum Role {
  BORROWER
  CUSTODIAN
}

model RevokedToken {
  jti       String   @id
  expiresAt DateTime
  createdAt DateTime @default(now())
}

model EquipmentItem {
  id           String        @id @default(cuid())
  assetTag     String        @unique
  name         String
  category     String
  modelNumber  String?
  serialNumber String        @unique
  description  String?
  condition    ItemCondition @default(GOOD)
  status       ItemStatus    @default(AVAILABLE)
  createdAt    DateTime      @default(now())
  updatedAt    DateTime      @updatedAt

  requests     BorrowRequest[]
  loans        Loan[]
  auditLogs    AuditLog[]

  @@index([status])
  @@index([category])
```

```
}

enum ItemCondition {
  NEW
  GOOD
  FAIR
  DAMAGED
}

enum ItemStatus {
  AVAILABLE
  RESERVED
  CHECKED_OUT
  MAINTENANCE
  RETIRED
}

model BorrowRequest {
  id              String        @id @default(cuid())
  borrowerId      String
  itemId          String
  status          RequestStatus @default(PENDING)
  requestNote     String?
  rejectionReason String?
  createdAt       DateTime      @default(now())
  updatedAt       DateTime      @updatedAt

  borrower        User          @relation(fields: [borrowerId], references: [id])
  item            EquipmentItem @relation(fields: [itemId], references: [id])
  loan            Loan?

  @@index([borrowerId])
  @@index([itemId])
  @@index([status])
}

enum RequestStatus {
  PENDING
  APPROVED
  REJECTED
  CANCELLED
  EXPIRED
}

model Loan {
  id              String         @id @default(cuid())
  requestId       String?        @unique
  itemId          String
  borrowerId      String
  custodianId     String
  status          LoanStatus     @default(ACTIVE)
  checkedOutAt    DateTime       @default(now())
  dueDate         DateTime
  returnedAt      DateTime?
  checkoutNotes   String?
  returnNotes     String?
  returnCondition ItemCondition?
  createdAt       DateTime       @default(now())
  updatedAt       DateTime       @updatedAt

  request         BorrowRequest? @relation(fields: [requestId], references: [id])
  item            EquipmentItem  @relation(fields: [itemId], references: [id])
  borrower        User           @relation("BorrowerLoans", fields: [borrowerId],
    references: [id])
  custodian       User           @relation("CustodianLoans", fields: [custodianId],
    references: [id])
  @@index([borrowerId])
```

```
  @@index([custodianId])
  @@index([itemId])
  @@index([status])
}

enum LoanStatus {
  ACTIVE
  RETURNED
  OVERDUE
}

model AuditLog {
  id            String   @id @default(cuid())
  entityType    String
  entityId      String
  action        String
  details       String?
  performedById String
  timestamp     DateTime @default(now())

  performedBy   User     @relation(fields: [performedById], references: [id])
  item          EquipmentItem? @relation(fields: [entityId], references: [id])

  @@index([entityType, entityId])
}
```

6.2 Seed Data Specifications

The idempotent seed script (`prisma/seed.ts`) generates:

• **Borrower 1:** `borrower@cpe.kmutt.ac.th` / `[local seed password omitted]` (Student ID: `67070501001`)

• **Borrower 2:** `borrower2@cpe.kmutt.ac.th` / `[local seed password omitted]` (Student ID: `67070501002`, for cross-user isolation tests)

• **Custodian:** `custodian@cpe.kmutt.ac.th` / `[local seed password omitted]` (Staff ID: `STAFF-001`)

• **Equipment Assets (6 Units)**: `CPE-OSC-001`: Rigol DS1054Z Digital Oscilloscope (`AVAILABLE`) `CPE-OSC-002`: Keysight InfiniiVision DSOX1204A (`AVAILABLE`) `CPE-RPI-001`: Raspberry Pi 4 Model B 4GB (`AVAILABLE`) `CPE-RPI-002`: Raspberry Pi 4 Model B 8GB (`CHECKED_OUT` on pre-seeded active loan) `CPE-FPG-001`: Terasic DE10-Lite FPGA Board (`AVAILABLE`) `CPE-LOG-001`: Saleae Logic 8 USB Analyzer (`MAINTENANCE`)

7. Product Definition of Done (DoD)

The Sprint 1 vertical slice is marked **Done** when all of the following criteria are verified:

1. **Full Domain Slice Functional**: Borrower logs in → browses catalog → requests available item (status becomes `RESERVED`). Borrower views pending request in `/my-requests`. Custodian logs in → reviews pending request in `/custodian/desk` → records checkout. Item status transitions to `CHECKED_OUT`, active `Loan` record is created, and 7-day `dueDate` is saved.

2. **Business Rules Verified**: Requesting an occupied item is rejected with HTTP 400 (`ITEM_NOT_AVAILABLE`). Exceeding quota of 3 active items is rejected with HTTP 400 (`EXCEEDS_ACTIVE_QUOTA`). Concurrent checkouts for the same asset are rejected with HTTP 409 Conflict.

3. **Security & Authorization Verified**: Unauthenticated requests to protected routes return HTTP 401. Borrowers attempting custodian actions return HTTP 403. Calling `POST /api/v1/auth/logout` revokes the specific token (`jti`), returning HTTP 401 on subsequent requests while keeping concurrent sessions active. Zero plaintext passwords in database or logs (bcrypt ≥ 10 salt rounds).

4. **Automated Testing Complete**: 100% of planned tests in `tests.md` pass with zero failures or skipped tests (`npm test`). Playwright E2E browser journeys pass cleanly (`npx playwright test`).

5. **Git & Integration Requirements**: Feature contracts committed before implementation. Features implemented on branches off `sprint-1-staging`, peer-reviewed, and merged. Reviewed release PR merged into `main` and tagged as `v0.1.0-sprint1`.

Source: docs/sprints/sprint-1/ui-spec.md

Sprint 1 UI Specification

EquipFlow Blue Matcha Design System

Document ID: `UI-SPEC-SPRINT-01` **Version:** `1.2.1` **Status:** Approved (Verified `v0.1.0-sprint1`) **Target Release:** `v0.1.0-sprint1`

1. Design System & Visual Language

EquipFlow features the **Blue Matcha Design System**, blending calming marine depth with earthy matcha botanical accents and creamy porcelain clarity. The visual language unites **Deep Ocean** navy structure, vibrant **Matcha Green** availability indicators, a soothing **Vanilla Milk** canvas, **Airy Blue** interactive cues, and **Frozen Aqua** subtle surface tints across desktop, tablet, and mobile displays.

1.1 Blue Matcha Color Tokens Palette

Token Name	Hex Code	Visual Role & Component Usage
`--color-deep-ocean`	`#31465A`	**Primary Brand & Structure**: Top navbar header, primary buttons (`.btn-primary`), primary headers, modal titles
`--color-matcha-green`	`#C7DFA3`	**Matcha Accent & Success**: `AVAILABLE` status badges, quota progress bar fill, success highlights
`--color-vanilla-milk`	`#FFFDF7`	**Warm Canvas & Surfaces**: Application page background (`body`), card surfaces, modal body background
`--color-airy-blue`	`#89B9E6`	**Interactive Accent & Status**: Button hover (`.btn-primary:hover`), active tab indicators, `RESERVED` status badge
`--color-frozen-aqua`	`#D9F0FF`	**Subtle Surface & Triage**: Table headers, read-only field surfaces, demo callout box, `CHECKED_OUT` badge
`--color-focus-ring`	`#31465A`	**High-Contrast Focus Indicator**: 2px solid Deep Ocean outline (`outline-offset: 2px`) for keyboard focus on light surfaces
`--color-focus-ring-dark`	`#FFFDF7`	**Inverted Focus Indicator**: 2px solid Vanilla Milk outline on dark surfaces (e.g. Deep Ocean navbar)
`--color-deep-ocean-dark`	`#1C2A37`	High-contrast deep slate text for body copy and headings on Vanilla Milk
`--color-muted-slate`	`#586D82`	Secondary text, field helper labels, timestamps, table column subtitles
`--color-border-subtle`	`#DCE8F2`	1px soft neutral border for cards, table dividers, form field boundaries
`--color-error-bg`	`#FEE2E2`	Validation error alert background
`--color-error-text`	`#991B1B`	High-contrast dark crimson error text
`--color-error-border`	`#DC2626`	1px red border applied to invalid input controls

1.2 Text & Non-Text Contrast Compliance (WCAG 2.1 AA / AAA)

1.2.1 Text Contrast Evaluation (WCAG 2.1 SC 1.4.3)

Every foreground/background color pair in the Blue Matcha palette is mathematically evaluated and verified under WCAG 2.1 relative luminance standards:

Foreground Color	Background Color	Evaluated Ratio	WCAG 2.1 Level	UI Application & Context
White (`#FFFFFF`)	Deep Ocean (`#31465A`)	**9.75:1**	**AAA** (≥ 7.0 : 1)	Top navbar title, primary button label (`.btn-primary`), modal header text
Deep Ocean (`#31465A`)	Vanilla Milk (`#FFFDF7`)	**9.58:1**	**AAA** (≥ 7.0 : 1)	Page body text, card titles, form labels, table body content
Deep Ocean (`#31465A`)	Frozen Aqua (`#D9F0FF`)	**8.29:1**	**AAA** (≥ 7.0 : 1)	Table headers (`th`), callout credentials card, `CHECKED_OUT` badge text
Deep Ocean (`#31465A`)	Matcha Green (`#C7DFA3`)	**6.74:1**	**AA** (≥ 4.5 : 1)	`AVAILABLE` status badge text, quota progress counter badge
Deep Ocean (`#31465A`)	Airy Blue (`#89B9E6`)	**4.71:1**	**AA** (≥ 4.5 : 1)	`RESERVED` status badge text, active tab indicators
Deep Ocean Dark (`#1C2A37`)	Vanilla Milk (`#FFFDF7`)	**14.38:1**	**AAA** (≥ 7.0 : 1)	Primary headings, high-contrast body text
Muted Slate (`#586D82`)	Vanilla Milk (`#FFFDF7`)	**5.26:1**	**AA** (≥ 4.5 : 1)	Field helper labels, timestamps, table column subtitles
Dark Crimson (`#991B1B`)	Soft Coral Red (`#FEE2E2`)	**6.80:1**	**AA** (≥ 4.5 : 1)	Form validation error alerts, maintenance badge text

1.2.2 Non-Text Contrast Evaluation for Focus Indicators & UI Components (WCAG 2.1 SC 1.4.11 ≥ 3 : 1)

Per WCAG 2.1 Success Criterion 1.4.11 (Non-text Contrast), visual focus indicators and active interactive control boundaries must achieve at least **3:1** contrast against adjacent background colors.

[!NOTE] Focus Indicator Contrast Evaluation While Airy Blue (`#89B9E6`, L = 0.45729) provides an elegant interactive accent, its contrast against light surfaces fails SC 1.4.11: Against Vanilla Milk (`#FFFDF7`): **2.03:1** (Fails ≥ 3 : 1) Against Pure White (`#FFFFFF`): **2.07:1** (Fails ≥ 3 : 1) Against Frozen Aqua (`#D9F0FF`): **1.76:1** (Fails ≥ 3 : 1) Therefore, EquipFlow designates **Deep Ocean (`#31465A`)** as the primary focus indicator token (`--color-focus-ring: #31465A`), providing outstanding contrast on all light UI surfaces. For elements on dark backgrounds (such as the top navigation bar), **Vanilla Milk (`#FFFDF7`)** or a high-contrast dual-tone ring is applied.

Focus Ring Token / Color	Adjacent Surface	Evaluated Ratio	WCAG 2.1 SC 1.4.11 (≥ 3 : 1)	Context & Component Usage
Deep Ocean (`#31465A`)	Vanilla Milk (`#FFFDF7`)	**9.58:1**	**Pass** (Exceeds 3:1)	Input fields, card buttons, general page `:focus-visible`
Deep Ocean (`#31465A`)	Pure White (`#FFFFFF`)	**9.75:1**	**Pass** (Exceeds 3:1)	Equipment cards, modal dialog inputs, table actions
Deep Ocean (`#31465A`)	Frozen Aqua (`#D9F0FF`)	**8.29:1**	**Pass** (Exceeds 3:1)	Triage table rows, callout interactive links
Deep Ocean (`#31465A`)	Matcha Green (`#C7DFA3`)	**6.74:1**	**Pass** (Exceeds 3:1)	Quota pill, status badges
Deep Ocean (`#31465A`)	Airy Blue (`#89B9E6`)	**4.71:1**	**Pass** (Exceeds 3:1)	Action buttons, tab controls

Focus Ring Token / Color	Adjacent Surface	Evaluated Ratio	WCAG 2.1 SC 1.4.11 (≥ 3 : 1)	Context & Component Usage
Vanilla Milk (`#FFFDF7`)	Deep Ocean Navbar (`#31465A`)	**9.58:1**	**Pass** (Exceeds 3:1)	Top navbar links and logout button `:focus-visible`
Airy Blue (`#89B9E6`)	Deep Ocean Navbar (`#31465A`)	**4.71:1**	**Pass** (Exceeds 3:1)	Top navbar active states and focus rings on dark nav

1.3 Status Badges (With Non-Color Cues)

Every status indicator combines distinct Blue Matcha color tokens with an explicit textual label and semantic icon to guarantee accessibility under WCAG 2.1 AA guidelines:

State	Badge Class	Visual Background / Text	Non-Color Icon & Label Cue
Available	`.badge-avail`	Matcha Green (`#C7DFA3` / text `#31465A`)	□ `Available`
Reserved	`.badge-res`	Airy Blue (`#89B9E6` / text `#31465A`)	□ `Reserved`
Checked Out	`.badge-out`	Frozen Aqua (`#D9F0FF` / text `#31465A`)	□ `Checked Out`
Maintenance	`.badge-maint`	Soft Coral Red (`#FEE2E2` / text `#991B1B`)	□ `Maintenance`
Pending Request	`.badge-pending`	Airy Blue (`#89B9E6` / text `#31465A`)	□ `Pending Approval`
Active Loan	`.badge-active`	Frozen Aqua (`#D9F0FF` / text `#31465A`)	□ `Active Loan`

2. Borrower Portal UI Specification

2.1 Login View (`/login`)

• **Layout:** Centered card (maximum width `450px`) on a warm Vanilla Milk backdrop (`#FFFDF7`).

• **Header Area:** Background: Deep Ocean (`#31465A`) with crisp white typography. Application Title: **EquipFlow — CPE Hardware Lending**. Subtitle: `Department of Computer Engineering`.

• **Form Controls:** Input: `Email Address` (`type="email"`, required, autofocus). Input: `Password` (`type="password"`, required). Primary Button: `"Sign In"` (`background: #31465A`, hover: `#89B9E6`, text: white). Disables and renders spinner (`"Signing in..."`) while request is in-flight.

• **Frozen Aqua Callout Box (`#D9F0FF`):** Below the login form, a prominent callout card styled with `background: #D9F0FF; border: 1px solid #B8E0FC; color: #31465A;` displaying local development credentials: Borrower: `borrower@cpe.kmutt.ac.th` / `[local seed password omitted]` Custodian: `custodian@cpe.kmutt.ac.th` / `[local seed password omitted]`

• **Error Handling:** If authentication fails, renders a dismissible red alert banner: `"Invalid email or password."`.

2.2 Top Navigation Bar (Global)

• **Visual Style:** Background: Deep Ocean (`#31465A`) with a subtle 2px bottom border in Airy Blue (`#89B9E6`). Brand Logo: White text with bold font. Active Navigation Links: Highlighted in Airy Blue (`#89B9E6`) with an active underline.

• **User Identity Section:** Authenticated user's `fullName` rendered in white. Role Badge rendered in **Matcha Green** (`background: #C7DFA3; color: #31465A; font-weight: bold;`) showing `[Borrower]` or `[Custodian]`. `"Logout"` Button: Outlined white button with hover fill.

2.3 Equipment Catalog View (`/catalog`)

• **Header Area:** Title: `Department Equipment Catalog` (`color: #31465A`). Subtitle: `Browse real-time hardware availability and submit borrowing requests.` Right Action: **Active Quota Pill** styled in Matcha Green: `<span class="badge" style="background-color: #C7DFA3; color: #31465A; font-size: 0.95rem;">  My Active Items: <strong>X / 3</strong> </span>`

• **Equipment Card Grid:** Responsive card layout (`background: #FFFFFF; border: 1px solid #DCE8F2; border-radius: 8px;`). **Card Header:** Asset Tag in Deep Ocean text and real-time status badge (`.badge-avail`, `.badge-res`, etc.). **Card Body:** Equipment Title (e.g. `Rigol DS1054Z Digital Oscilloscope`), Category, Model, Condition. **Card Footer:** `"View Details"` button (outlined Airy Blue `#89B9E6`, text `#31465A`). `"Request Item"` button: Active state: Deep Ocean (`#31465A`, hover `#89B9E6`) when status is `AVAILABLE` and Borrower Active Quota < 3. Disabled state: Soft gray (`#E2E8F0`) with explicit text: `"Reserved"`, `"Checked Out"`, or `"Limit Reached (3/3)"`.

• **Equipment Detail & Request Modal:** Modal header styled in Deep Ocean (`#31465A`) with white text. Displays specifications, manufacturer serial number, and condition. Optional Textarea: `"Academic Purpose / Course Project Notes"` (max 250 characters). Action: `"Confirm Request"` button (`#31465A`). Disables immediately on click to prevent duplicate submissions.

2.4 My Requests & Loans View (`/my-requests`)

• Accessible to Borrowers from top navigation.

• **Quota Progress Banner:** Card styled with a Matcha Green accent border (`border-left: 4px solid #C7DFA3;`). Progress Bar: Track in Frozen Aqua (`#D9F0FF`), filled bar in Matcha Green (`#C7DFA3`) indicating active load (e.g. `1 of 3 items in use`).

• **Active Loans Section:** Table headers styled with Frozen Aqua tint (`#D9F0FF`, text `#31465A`). Columns: `Asset Tag`, `Equipment Name`, `Checked Out At`, `Due Date`, `Status`. `Due Date` highlighted in bold with a clock cue.

• **Pending Requests Section:** Columns: `Asset Tag`, `Equipment Name`, `Requested Date`, `Purpose Notes`, `Status`. Status rendered with Airy Blue badge (`.badge-pending`).

• **Empty State:** Illustrative empty state card with Frozen Aqua tint: `"You have no active loans or requests. Browse the catalog to borrow equipment."` with a primary button linking to `/catalog`.

3. Custodian Operations Desk UI Specification

3.1 Custodian Desk View (`/custodian/desk`)

• **Access Boundary:** Restricted strictly to users authenticated with role `CUSTODIAN`. Borrowers attempting access are redirected with an unauthorized toast notice.

• **Header Area:** Title: `Custodian Triage & Checkout Desk` (`color: #31465A`). Subtitle: `Review student requests, verify physical custody, and record checkouts.` Right Action: `"Refresh Queue"` button (Airy Blue `#89B9E6` with sync icon).

• **Triage Work Area: Pending Requests Queue (Primary Work Table):** Columns: `Requested Time`, `Student Name`, `Student ID`, `Asset Tag`, `Equipment Name`, `Notes`, `Action`. Table header styled in Frozen Aqua (`#D9F0FF`, text `#31465A`). Student ID styled as a subtle code badge. Action Column: Primary button `"Approve & Checkout"` (`background: #31465A`, hover `#89B9E6`, text: white). Clicking `"Approve & Checkout"` opens the **Checkout Confirmation Modal**. **Active Loans Monitor:** Summarizes all currently checked out items across the department. Columns: `Asset Tag`, `Equipment Name`, `Borrower Name`, `Checked Out At`, `Due Date` (7-day deadline), `Accountable Custodian`.

3.2 Checkout Confirmation Modal

• Modal header styled in Deep Ocean (`#31465A`) with white text.

• Displays verified Borrower identity: `Student Borrower One (67070501001).`

• Displays verified Equipment identity: `CPE-OSC-001 (Rigol DS1054Z).`

• **7-Day Deadline Callout:** Boxed callout styled in **Frozen Aqua Accent** (`background: #D9F0FF; color: #31465A; border: 1px solid #B8E0FC; border-radius: 6px; padding: 12px;`): `Scheduled Due Date: 19 November 2026 (7 days from today)`

• Optional Input: `Checkout Staff Notes` (e.g. "Verified student ID card. Handed over with power cord.").

• Action Buttons: `"Cancel"` and `"Confirm Physical Checkout"`.

• Submit Button states: Disables immediately on click and displays `"Processing checkout..."` to prevent duplicate submissions.

4. Responsive Breakpoints & Accessibility

4.1 Responsive Layout Rules

Viewport	Screen Width	Layout Adjustments
Desktop	≥ 1024px	3-column equipment card grid on `/catalog`. Full data tables with all columns visible on `/custodian/desk` and `/my-requests`.
Tablet	768px to 1023px	2-column equipment card grid. Tables remain full width with horizontally scrollable container if needed.
Mobile	< 768px	1-column stacked equipment cards. Top navbar collapses into accessible hamburger menu. Tables display essential columns (`Asset Tag`, `Status`, `Action`) with secondary details stacked vertically.

4.2 Accessibility Standards (WCAG 2.1 AA)

• **Keyboard Navigation:** All interactive elements (`<button>`, `<a>`, `<input>`, `<select>`) support tab navigation in natural DOM order.

• **Visible Focus Rings:** High-contrast 2px solid focus ring in Deep Ocean (`--color-focus-ring: #31465A`, `outline: 2px solid #31465A; outline-offset: 2px;`) applied to active elements upon keyboard `:focus-visible` across all light surfaces (yielding **9.58:1** against Vanilla Milk `#FFFDF7`, **9.75:1** against White `#FFFFFF`, and **8.29:1** against Frozen Aqua `#D9F0FF`, strictly satisfying WCAG 2.1 SC 1.4.11 ≥ 3 : 1). For interactive elements located on dark surfaces (such as top navbar links), an inverted focus ring in Vanilla Milk (`--color-focus-ring-dark: #FFFDF7`, **9.58:1**) or dual-tone outline is applied.

• **Zero Horizontal Scroll:** Pages contain zero horizontal scrollbar at standard viewport widths (375px, 768px, 1280px).

• **Non-Color Dependency:** All status indicators accompany color badges with explicit text labels and semantic icons.

• **Color Contrast:** All body and header text pairs achieve contrast ratios ≥ 4.5 : 1 (AA) and ≥ 7 : 1 (AAA), fully certified in Section 1.2.

5. Shared Form, Error & Loading Behaviors

1. **Button Debounce & Loading Spinners:** All mutating submit buttons disable upon click and render an inline loading spinner to eliminate duplicate submissions caused by rapid clicks.

2. **Form Value Preservation:** Client validation failures retain all user-entered form data (e.g., purpose notes, checkout remarks) across recoverable validation errors.

3. **API Failure Feedback:** If a backend request fails with HTTP 400, 401, 403, or 409, the UI traps the JSON error envelope and displays the human-readable `error.message` in a dismissible alert banner.

4. **Skeleton Placeholders:** Initial page loads query APIs asynchronously while rendering pulsing Frozen Aqua skeleton cards (`#D9F0FF`) to prevent Cumulative Layout Shift (CLS).

Source: docs/sprints/sprint-1/api-spec.md

Sprint 1 REST API Specification

EquipFlow Hardware Inventory & Borrowing Endpoints

Document ID: `API-SPEC-SPRINT-01` **Version:** `1.0.0` **Status:** Approved (Verified `v0.1.0-sprint1`) **Target Release:** `v0.1.0-sprint1`

1. Overview & General Conventions

This document specifies the authoritative REST API contract for EquipFlow Sprint 1. All endpoints adhere to standard HTTP semantics, consume and produce JSON payloads, enforce stateless JWT authentication via Bearer tokens, and apply strict role-based access control.

1.1 Authentication & Authorization

• **Bearer Token Identification:** Authenticated requests must pass the JWT token in the HTTP Authorization header: `Authorization: Bearer <jwt-token>`

• **Roles:** `BORROWER`: Department student, faculty, or researcher borrowing equipment. `CUSTODIAN`: Department laboratory technician / equipment administrator.

• **HTTP Status Code Conventions:** `200 OK`: Request succeeded. `201 Created`: Resource successfully created. `400 Bad Request`: Payload validation failure or business quota violation (`EXCEEDS_ACTIVE_QUOTA`). `401 Unauthorized`: Missing, expired, malformed, or revoked token. `403 Forbidden`: Authenticated user lacks required role or is attempting cross-borrower access. `404 Not Found`: Target resource does not exist. `409 Conflict`: Concurrency conflict (e.g. concurrent checkout of the same asset). `500 Internal Server Error`: Unhandled server exception.

1.2 Standard Error Response Shape

All error responses across all endpoints adhere strictly to a standardized JSON error envelope:

1.3 Absence of DELETE Endpoints

In accordance with inventory tracking integrity and course specifications, **no DELETE endpoints exist** in Sprint 1 (`DELETE /api/v1/items/:id` and `DELETE /api/v1/requests/:id` are not implemented). Items transition through operational states (`AVAILABLE`, `RESERVED`, `CHECKED_OUT`, `MAINTENANCE`), and borrowing records form an append-only audit trail.

2. Authentication Endpoints

2.1 User Login

Authenticates registered department credentials and issues a signed JWT Bearer token.

• **Method:** `POST`

• **Path:** `/api/v1/auth/login`

• **Allowed Roles:** Public / Anonymous

• **Request Headers:** `Content-Type: application/json`

Request Body

Success Response: `200 OK`

Error Responses

• `401 Unauthorized`: `{ "success": false, "error": { "code": "INVALID_CREDENTIALS", "message": "Invalid email or password." }, "meta": { "timestamp": "2026-11-12T18:00:00.000Z" } }`

2.2 Get Current User Identity

Retrieves the profile and role of the currently authenticated user.

• **Method:** `GET`

• **Path:** `/api/v1/auth/me`

• **Allowed Roles:** `BORROWER`, `CUSTODIAN`

• **Request Headers:** `Authorization: Bearer <token>`

Success Response: `200 OK`

Error Responses

• `401 Unauthorized` (Token missing, expired, or revoked): `{ "success": false, "error": { "code": "TOKEN_REVOKED", "message": "This session has been logged out." }, "meta": { "timestamp": "2026-11-12T18:00:00.000Z" } }`

2.3 User Logout & Per-Token Revocation

Invalidates the current session's Bearer token on the server side by persisting its `jti` to the `RevokedToken` table.

• **Method:** `POST`

• **Path:** `/api/v1/auth/logout`

• **Allowed Roles:** `BORROWER`, `CUSTODIAN`

• **Request Headers:** `Authorization: Bearer <token>`

Success Response: `200 OK`

3. Equipment Catalog Endpoints

3.1 List Equipment Catalog

Returns all serialized equipment assets with real-time availability statuses.

• **Method:** `GET`

• **Path:** `/api/v1/items`

• **Allowed Roles:** `BORROWER`, `CUSTODIAN`

• **Request Headers:** `Authorization: Bearer <token>`

Success Response: `200 OK`

3.2 Get Equipment Item Detail

Returns complete technical specifications, manufacturer serial number, and physical condition.

• **Method:** `GET`

• **Path:** `/api/v1/items/:id`

• **Allowed Roles:** `BORROWER`, `CUSTODIAN`

• **Request Headers:** `Authorization: Bearer <token>`

Success Response: `200 OK`

Error Responses

• `404 Not Found`: `{ "success": false, "error": { "code": "ITEM_NOT_FOUND", "message": "Equipment asset with ID non_existent_id was not found." }, "meta": { "timestamp": "2026-11-12T18:00:00.000Z" } }`

4. Borrower Request Endpoints

4.1 Submit Borrow Request

Submits a reservation request for an available equipment item. The request is processed inside an atomic transaction: verifies item is `AVAILABLE`, verifies borrower active quota < 3, transitions item status to `RESERVED`, and creates a `BorrowRequest` in `PENDING` status.

• **Method:** `POST`

• **Path:** `/api/v1/requests`

• **Allowed Roles:** `BORROWER` (Borrowers only)

• **Request Headers:** `Authorization: Bearer <token>`, `Content-Type: application/json`

Request Body

Success Response: `201 Created`

Error Responses

• `400 Bad Request` (Item not available): `{ "success": false, "error": { "code": "ITEM_NOT_AVAILABLE", "message": "Equipment item CPE-OSC-001 is already reserved or checked out." }, "meta": { "timestamp": "2026-11-12T18:30:00.000Z" } }`

• `400 Bad Request` (Active quota exceeded): `{ "success": false, "error": { "code": "EXCEEDS_ACTIVE_QUOTA", "message": "Cannot submit request: active loans and requests combined cannot exceed 3." }, "meta": { "timestamp": "2026-11-12T18:30:00.000Z" } }`

• `403 Forbidden` (Custodian attempting borrower request): `{ "success": false, "error": { "code": "FORBIDDEN_ROLE", "message": "Borrow requests can only be submitted by Borrower accounts." }, "meta": { "timestamp": "2026-11-12T18:30:00.000Z" } }`

4.2 Get My Requests & Loans

Retrieves the authenticated borrower's personal pending requests and active loans. Strictly isolated to `req.user.userId`.

• **Method:** `GET`

• **Path:** `/api/v1/requests/my`

• **Allowed Roles:** `BORROWER` (Borrowers only)

• **Request Headers:** `Authorization: Bearer <token>`

Success Response: `200 OK`

5. Custodian Operations Endpoints

5.1 List Pending Requests Triage Queue

Retrieves all pending borrow requests across all borrowers, ordered chronologically, and active department loans for the custodian monitor.

• **Method:** `GET`

• **Path:** `/api/v1/custodian/requests`

• **Allowed Roles:** `CUSTODIAN` (Custodians only)

• **Request Headers:** `Authorization: Bearer <token>`

Success Response: `200 OK`

Error Responses

• `403 Forbidden` (Borrower accessing custodian triage): `{ "success": false, "error": { "code": "FORBIDDEN_ROLE", "message": "Access denied: requires CUSTODIAN role." }, "meta": { "timestamp": "2026-11-12T18:35:00.000Z" } }`

5.2 Execute Equipment Checkout

Records the physical handover of equipment against a pending request. Executed inside an atomic transaction: validates request is `PENDING`, validates item is `RESERVED`, conditionally transitions both records, sets request to `APPROVED`, creates an active `Loan` record, calculates `dueDate = checkedOutAt + 7 days`, and records `custodianId` from the authenticated session.

• **Method:** `POST`

• **Path:** `/api/v1/custodian/checkouts`

• **Allowed Roles:** `CUSTODIAN` (Custodians only)

• **Request Headers:** `Authorization: Bearer <token>`, `Content-Type: application/json`

Request Body

Success Response: `201 Created`

Error Responses

• `409 Conflict` (Concurrent checkout attempt or item state changed): `{ "success": false, "error": { "code": "CONCURRENT_CHECKOUT_CONFLICT", "message": "Checkout failed: the request is not pending or item is not reserved." }, "meta": { "timestamp": "2026-11-12T18:40:00.000Z" } }`

• `403 Forbidden` (Borrower attempting checkout): `{ "success": false, "error": { "code": "FORBIDDEN_ROLE", "message": "Access denied: requires CUSTODIAN role." }, "meta": { "timestamp": "2026-11-12T18:40:00.000Z" } }`

Source: docs/sprints/sprint-1/tests.md

Sprint 1 Test Plan & Traceability Matrix

EquipFlow Hardware Inventory & Borrowing Verification

Document ID: `TEST-PLAN-SPRINT-01` **Version:** `1.1.0` **Status:** Approved (Verified `v0.1.0-sprint1`)
Target Release: `v0.1.0-sprint1`

1. Test Strategy & Quality Plan

This document establishes the comprehensive Test Driven Development (TDD) and Test Driven Design (Test DD) plan for Sprint 1. In accordance with the CPE 334 engineering baseline, all domain rules, persistence integrity, access boundaries, and end-to-end user journeys are verified through automated test suites:

1. **Backend API & Integration Tests** (`server/tests/sprint-1/` via Vitest & Supertest): Validating REST contracts, Prisma queries, transaction boundaries, optimistic locking, role authorization, and validation rules. Real database integration running against isolated SQLite test databases (`DATABASE_URL="file:./test.db"`).

2. **Frontend UI Component Tests** (`client/tests/sprint-1/` via Vitest & React Testing Library): Validating component rendering, form state management, role-based controls, button debouncing, and empty/error states.

3. **End-to-End (E2E) Browser Tests** (`e2e/sprint-1/` via Playwright): Multi-role user journeys across Borrower and Custodian logins, catalog browsing, request submission, and custodian checkout. Responsive viewport verification across desktop (1280px), tablet (768px), and mobile (375px), asserting zero horizontal scroll and visible keyboard focus rings.

4. **TDD Red-to-Green Evidence Strategy**: For critical domain rules (`BR-01` availability, `BR-02` concurrency conflict, `BR-04` active quota limit ≤ 3, and `FR-03` token revocation), tests are written first. Terminal output demonstrating the initial test failure (Red) is captured before implementing the service layer logic, followed by passing execution (Green).

2. Requirement & Acceptance Criteria Traceability Matrix

Test ID	Type	Requirement / AC	What It Tests	Expected Result	Automated Test File	Final Status
API-01-01	API	FR-01 / AC-AUTH-01	User login with valid email and password	Returns HTTP 200 with signed JWT Bearer token and sanitized user profile (no passwordHash)	`server/tests/sprint-1/auth.api.test.ts`	**Passed**
API-01-02	API	FR-01 / AC-AUTH-02	User login with invalid password	Returns HTTP 401 with error code `INVALID_CREDENTIALS`	`server/tests/sprint-1/auth.api.test.ts`	**Passed**
API-01-03	API	FR-01 / AC-AUTH-02	User login with non-existent email	Returns HTTP 401 with error code `INVALID_CREDENTIALS`	`server/tests/sprint-1/auth.api.test.ts`	**Passed**
API-01-04	API	FR-02 / AC-AUTH-03	Fetch authenticated identity via `/api/v1/auth/me`	Returns HTTP 200 with matching authenticated user ID, email, and role	`server/tests/sprint-1/auth.api.test.ts`	**Passed**
API-01-05	API	FR-03 / AC-AUTH-04	User logout via `POST /api/v1/auth/logout`	Returns HTTP 200; token `jti` persisted to `RevokedToken` table	`server/tests/sprint-1/auth.api.test.ts`	**Passed**
API-01-06	API	FR-03 / AC-AUTH-04	Request protected endpoint using revoked token	Returns HTTP 401 Unauthorized with code `TOKEN_REVOKED`	`server/tests/sprint-1/auth.api.test.ts`	**Passed**
API-01-07	API	FR-03 / AC-AUTH-05	Two concurrent sessions for same user; logout Session 1	Session 1 returns HTTP 401; concurrent Session 2 continues to return HTTP 200	`server/tests/sprint-1/auth.api.test.ts`	**Passed**
API-01-08	API	FR-04 / AC-AUTH-06	Request protected endpoint without Authorization header	Returns HTTP 401 Unauthorized with code `UNAUTHENTICATED`	`server/tests/sprint-1/auth.api.test.ts`	**Passed**
API-01-09	API	FR-04 / AC-AUTH-07	Borrower attempts to access `/api/v1/custodian/requests`	Returns HTTP 403 Forbidden with code `FORBIDDEN_ROLE`	`server/tests/sprint-1/auth.api.test.ts`	**Passed**
API-01-10	API	NFR-01 / AC-AUTH-08	Verify password hashing in database	`passwordHash` string in DB begins with `$2a$` or `$2b$` and salt rounds ≥ 10	`server/tests/sprint-1/auth.api.test.ts`	**Passed**
API-01-11	API	FR-05 / AC-CAT-01	Browse catalog via `GET /api/v1/items`	Returns HTTP 200 with array of equipment items and accurate availability statuses	`server/tests/sprint-1/catalog.api.test.ts`	**Passed**
API-01-12	API	FR-06 / AC-CAT-02	Fetch single item details via `/api/v1/items/:id`	Returns HTTP 200 with technical description, serial number, and condition	`server/tests/sprint-1/catalog.api.test.ts`	**Passed**
API-01-13	API	FR-06 / AC-CAT-02	Fetch non-existent item detail ID	Returns HTTP 404 Not Found with code `ITEM_NOT_FOUND`	`server/tests/sprint-1/catalog.api.test.ts`	**Passed**
API-01-14	API	FR-09 / AC-REQ-01	Borrower submits request for `AVAILABLE` item	Returns HTTP 201; request status `PENDING`; item status updated to `RESERVED`	`server/tests/sprint-1/requests.api.test.ts`	**Passed**

Test ID	Type	Requirement / AC	What It Tests	Expected Result	Automated Test File	Final Status
API-01-15	API	BR-01 / AC-REQ-02	Borrower submits request for `RESERVED` or `CHECKED_OUT` item	Returns HTTP 400 Bad Request with code `ITEM_NOT_AVAILABLE`; item status unchanged	`server/tests/sprint-1/requests.api.test.ts`	**Passed**
API-01-16	API	BR-04 / AC-REQ-03	Borrower submits request when active quota ≥ 3	Returns HTTP 400 Bad Request with code `EXCEEDS_ACTIVE_QUOTA`; no request created	`server/tests/sprint-1/requests.api.test.ts`	**Passed**
API-01-17	API	BR-02 / AC-REQ-04	Concurrent requests for the same available item	Exactly 1 request succeeds (HTTP 201); competing request rejected with HTTP 409 Conflict (`CONCURRENT_REQUEST_CONFLICT`)	`server/tests/sprint-1/requests.api.test.ts`	**Passed**
API-01-18	API	FR-10 / AC-REQ-05	Borrower queries `/api/v1/requests/my`	Returns HTTP 200 with borrower's personal requests and active loans	`server/tests/sprint-1/requests.api.test.ts`	**Passed**
API-01-19	API	FR-11 / AC-REQ-06	Cross-borrower data isolation check	Borrower A cannot view Borrower B's requests/loans; query results strictly scoped	`server/tests/sprint-1/requests.api.test.ts`	**Passed**
API-01-20	API	FR-13 / AC-CHK-01	Custodian fetches pending triage queue	Returns HTTP 200 with pending requests, borrower details, item tags, and active loans	`server/tests/sprint-1/checkout.api.test.ts`	**Passed**
API-01-21	API	FR-14 / AC-CHK-02	Custodian checks out reserved item	Returns HTTP 201; item status → `CHECKED_OUT`; request status → `APPROVED`; Loan created	`server/tests/sprint-1/checkout.api.test.ts`	**Passed**
API-01-22	API	BR-05 / AC-CHK-03	Due date calculation check on checkout	Loan record stores `dueDate` exactly 7 days after `checkedOutAt`	`server/tests/sprint-1/checkout.api.test.ts`	**Passed**
API-01-23	API	BR-08 / AC-CHK-04	Accountable custodian attribution check	Loan record stores `custodianId` matching authenticated custodian session	`server/tests/sprint-1/checkout.api.test.ts`	**Passed**
API-01-24	API	BR-02 / AC-CHK-05	Duplicate checkout attempt on already checked out item	Returns HTTP 409 Conflict with code `CONCURRENT_CHECKOUT_CONFLICT`	`server/tests/sprint-1/checkout.api.test.ts`	**Passed**
API-01-25	API	BR-02 / AC-CHK-06	Concurrent checkout calls for same pending request	Exactly 1 checkout succeeds (HTTP 201); second call returns HTTP 409 Conflict	`server/tests/sprint-1/checkout.api.test.ts`	**Passed**
API-01-26	API	BR-03 / AC-CHK-07	Borrower attempts to invoke checkout endpoint	Returns HTTP 403 Forbidden with code `FORBIDDEN_ROLE`	`server/tests/sprint-1/checkout.api.test.ts`	**Passed**

Test ID	Type	Requirement / AC	What It Tests	Expected Result	Automated Test File	Final Status
API-01-27	API	BR-04 / AC-REQ-03	Concurrent requests from same borrower exceeding	Exactly 1 request succeeds (total quota	`server/tests/sprint-1/requests.api.tes`	**Passed**

Test ID	Type	Requirement / AC	What It Tests	Expected Result	Automated Test File	Final Status
			quota (starts at 2, submits 2 items concurrently)	reaches 3); second request rejected with HTTP 400 (`EXCEEDS_ACTIVE_QUOTA`)	`t.ts`	
UI-01-01	UI	AC-AUTH-01	Login form rendering, credentials input & submit	Form renders email/password; button shows loading state during submit	`client/tests/sprint-1/Login.test.tsx`	**Passed**
UI-01-02	UI	AC-AUTH-02	Invalid login alert banner rendering	Displays red alert banner upon HTTP 401; preserves entered email address	`client/tests/sprint-1/Login.test.tsx`	**Passed**
UI-01-03	UI	AC-CAT-01	Catalog grid rendering & status badges	Cards render name, category, and readable Blue Matcha status badges	`client/tests/sprint-1/Catalog.test.tsx`	**Passed**
UI-01-04	UI	AC-CAT-02	Detail modal opens with specs and serial number	Clicking item opens modal with full description, serial, condition, and availability	`client/tests/sprint-1/Catalog.test.tsx`	**Passed**
UI-01-05	UI	AC-REQ-01	Request button disabled when item is unavailable	"Request to Borrow" button is disabled with gray badge if status != AVAILABLE	`client/tests/sprint-1/Catalog.test.tsx`	**Passed**
UI-01-06	UI	AC-REQ-03	Request button disabled when borrower active quota = 3	Quota badge displays "3/3"; request buttons disable with "Limit Reached"	`client/tests/sprint-1/Catalog.test.tsx`	**Passed**
UI-01-07	UI	AC-REQ-05	My Requests view displays pending requests & loans	Tables render active loans and pending requests with 7-day due date	`client/tests/sprint-1/MyRequests.test.tsx`	**Passed**
UI-01-08	UI	AC-CHK-01	Custodian triage queue table rendering	Renders pending requests table with borrower name and "Approve & Checkout" button	`client/tests/sprint-1/CustodianDesk.test.tsx`	**Passed**
E2E-01-01	E2E	AC-REQ-01, AC-CHK-02	Full Domain Journey: Borrower request to Custodian checkout	Borrower logs in, requests item -> Custodian logs in, checks out item -> DB verified	`e2e/sprint-1/domain-journey.spec.ts`	**Passed**
E2E-01-02	E2E	AC-REQ-02, AC-CHK-05	Meaningful Business Rule Rejection: Duplicate booking/checkout	Attempt to request or checkout occupied asset displays human-readable error banner	`e2e/sprint-1/rejection-rules.spec.ts`	**Passed**
E2E-01-03	E2E	AC-AUTH-06, AC-AUTH-07	Access Control: Borrower forbidden from Custodian desk	Direct navigation to `/custodian/desk` as Borrower redirects with forbidden notice	`e2e/sprint-1/access-control.spec.ts`	**Passed**
E2E-01-04	E2E	§4.1, §4.2	Viewport layout audit (375px/768px/1280px) and focus rings	Real browser verification of zero horizontal scroll and visible keyboard focus rings	`e2e/sprint-1/layout-a11y.spec.ts`	**Passed**

3. Test Execution Guidelines

Seed-State Contract & Deterministic Re-Runnability: Both the backend test runner (`server/vitest.config.ts` via `server/tests/global-setup.ts`) and the Playwright E2E runner (`playwright.config.ts` via `e2e/sprint-1/global-setup.ts` and `global-teardown.ts`) provision a fresh, disposable SQLite test database (`DATABASE_URL="file:./test.db"`), apply migrations (`npx prisma migrate deploy`), and seed baseline fixtures (`npx prisma db seed`) before running tests, then remove `prisma/test.db` upon teardown. Mutating E2E scenarios create dedicated test users and equipment items and clean up strictly their own created IDs.

3.1 Backend Integration Tests

Execute all backend API and domain rule tests:

3.2 Frontend Component Tests

Execute React Testing Library component tests:

3.3 End-to-End (E2E) Playwright Tests

Execute browser automation test journeys:

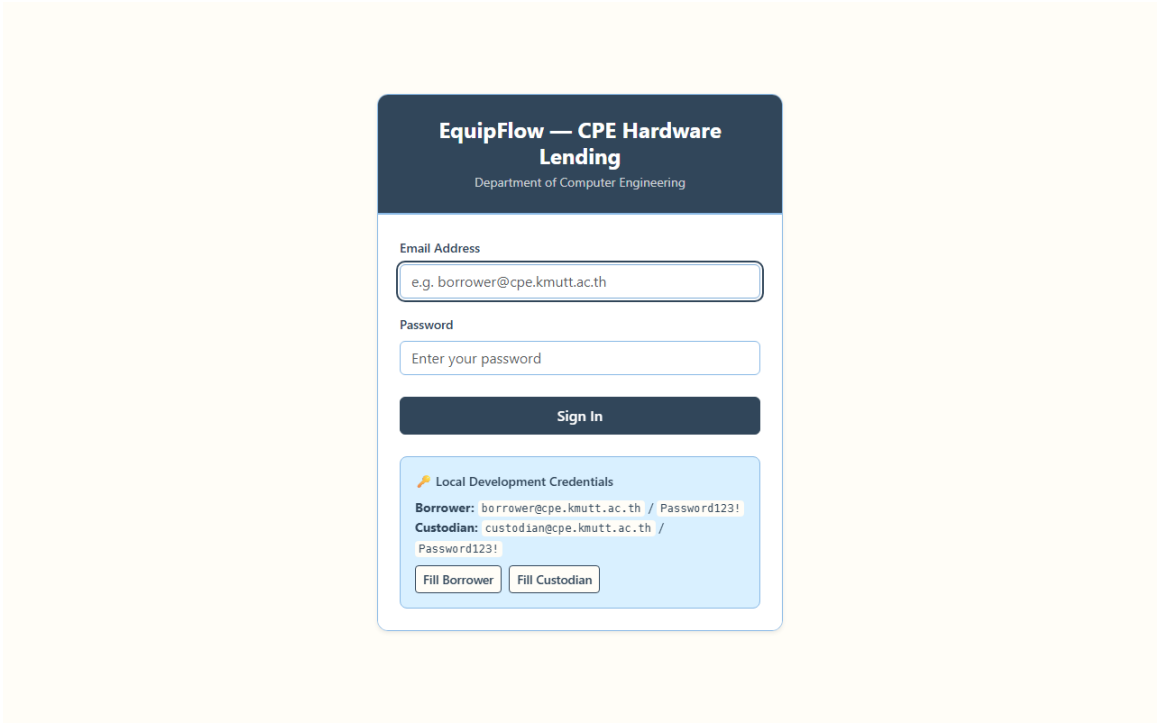
3.4 Full Test Suite Execution

Execute the entire test battery across backend, frontend, and E2E:

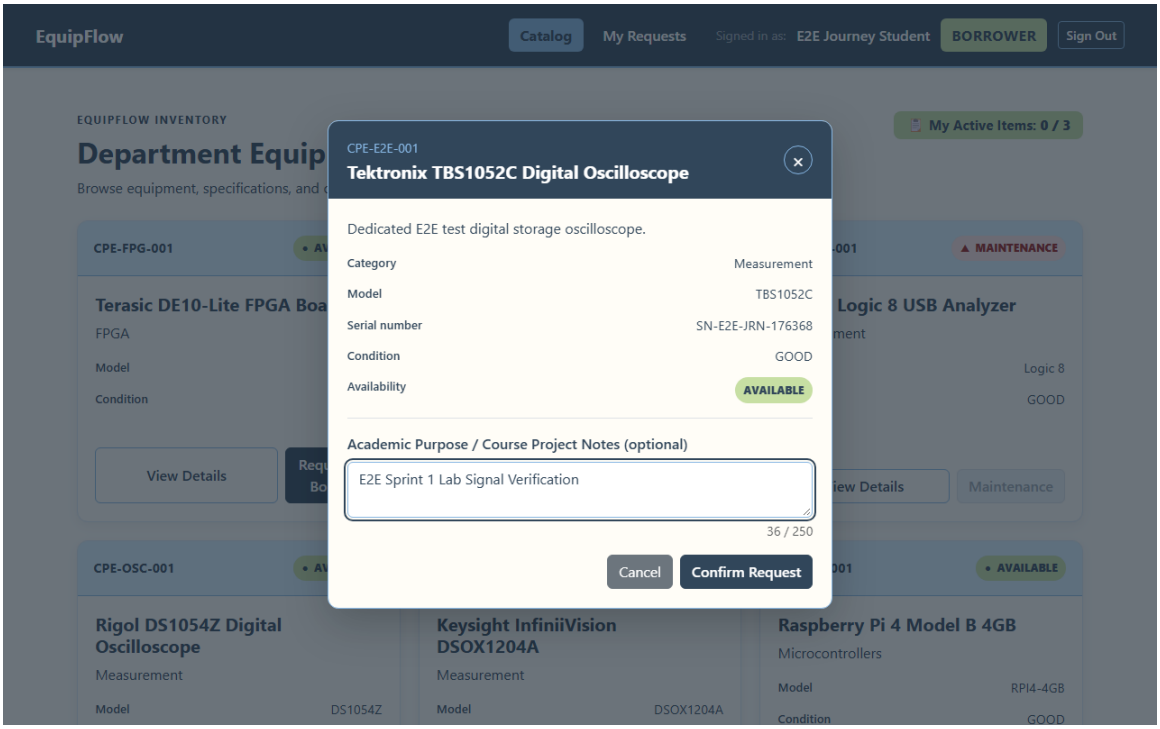
Part F · Working Sprint 1 and verification

F.1 · Demonstration and persisted result

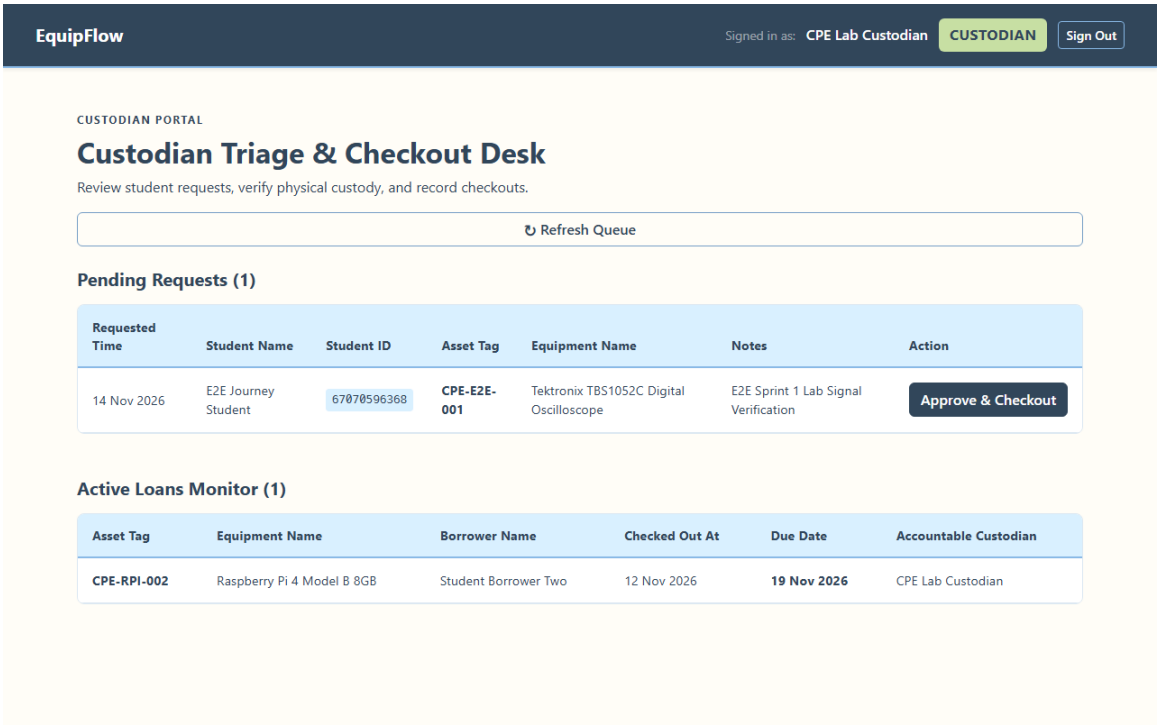
Source screenshots were committed in PR #16 from the author's test run. The test code commit is 63e0426. The screenshots in this draft are staging evidence; record any replacement captures against the submitted release.



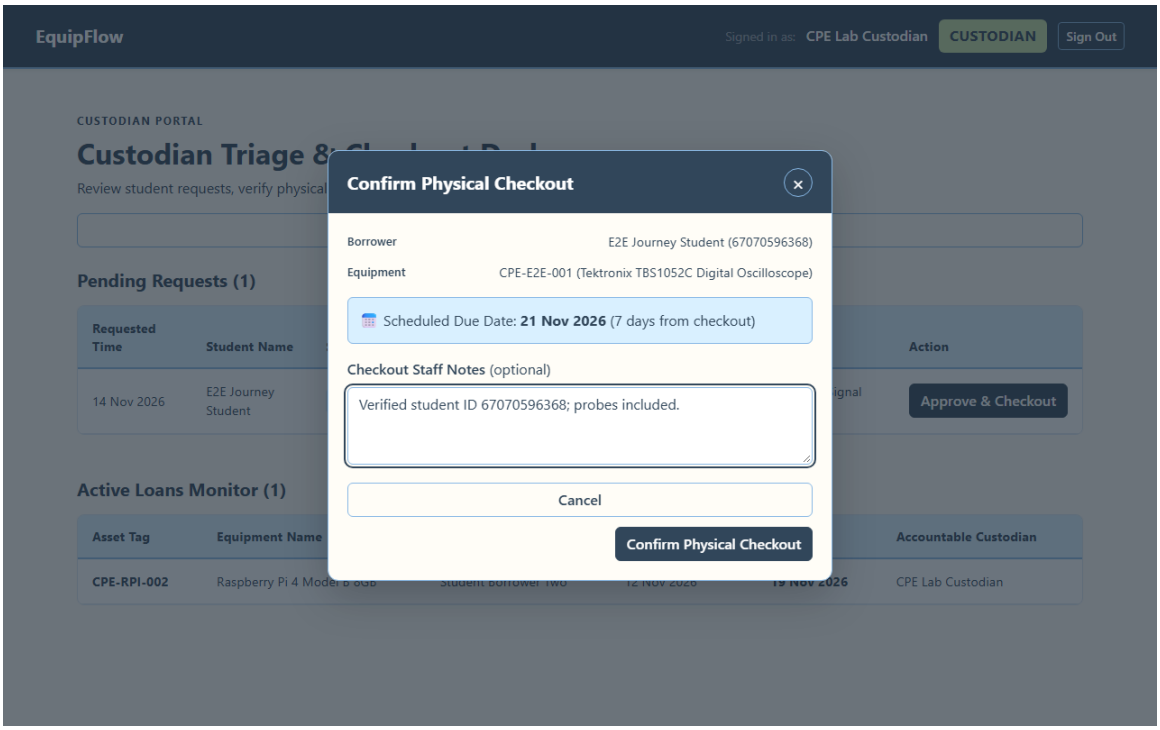
F-01 / AC-AUTH-01. Starting state: signed out. Open the app. The sign-in screen appears; the corresponding test enters local seeded credentials.



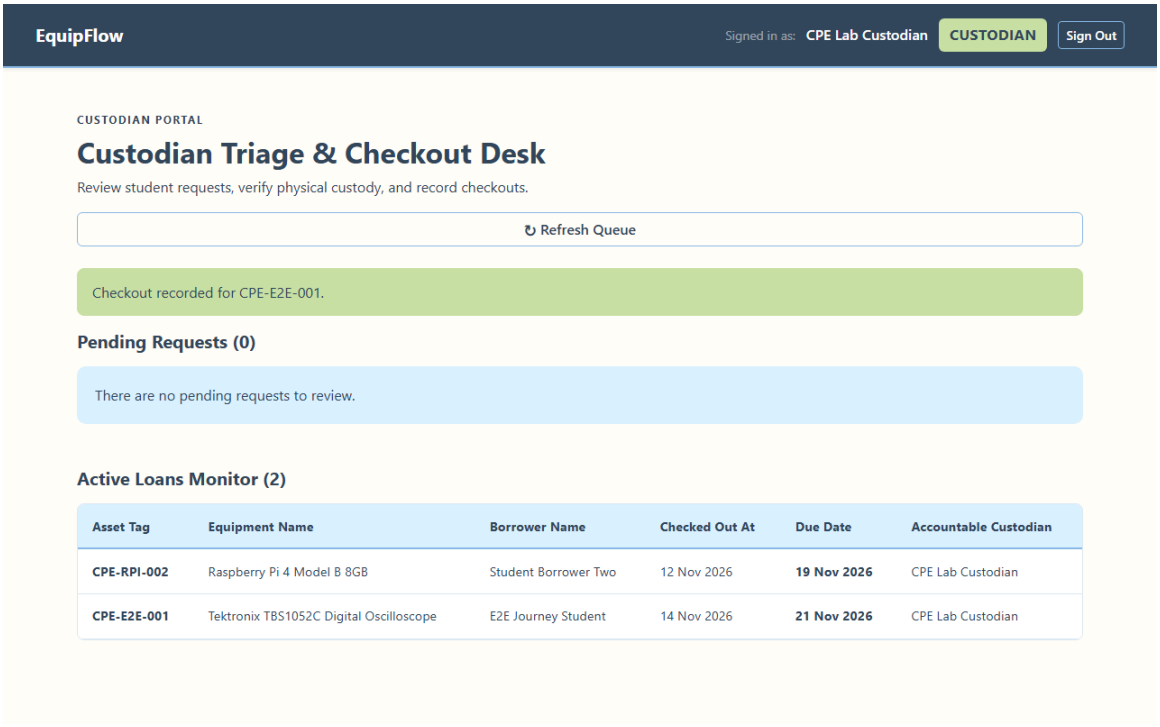
F-03 / AC-REQ-01. An authenticated Borrower selects the available test oscilloscope and confirms a request. Expected and recorded: the request persists as PENDING and the asset becomes RESERVED.



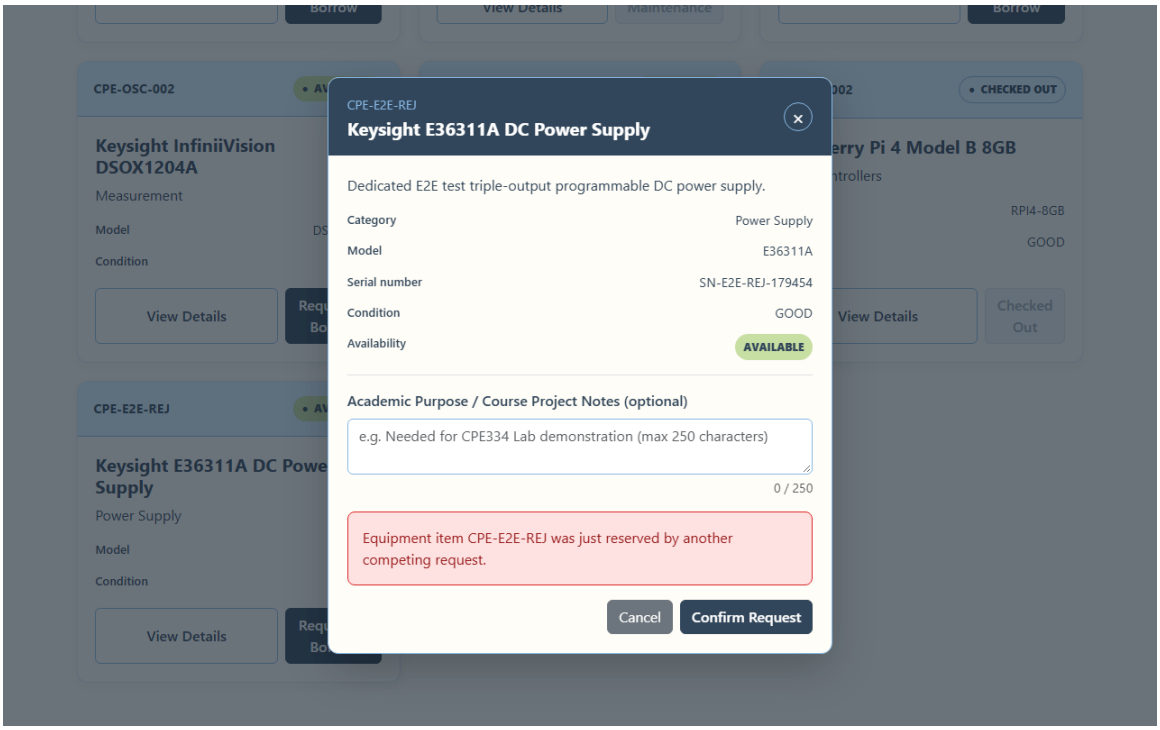
F-04 / AC-CHK-01. Custodian signs in after the Borrower signs out, opens the queue, and retrieves the pending request for CPE-E2E-001.



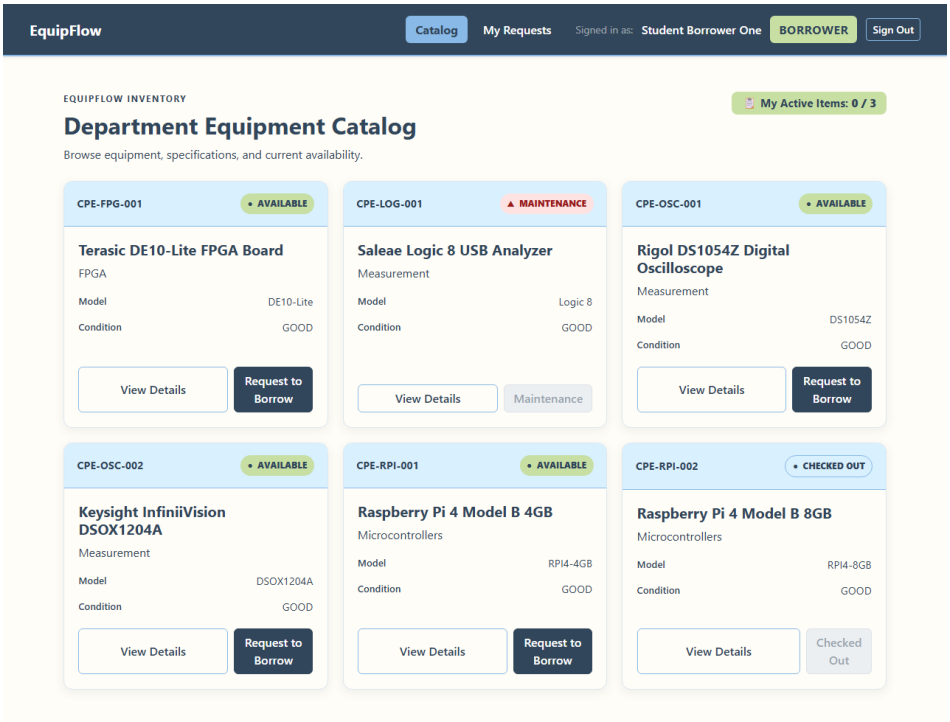
F-04 / AC-CHK-02–04. Custodian opens checkout on that pending request and confirms. The browser journey and Prisma assertions verify the approved request and checked-out item.



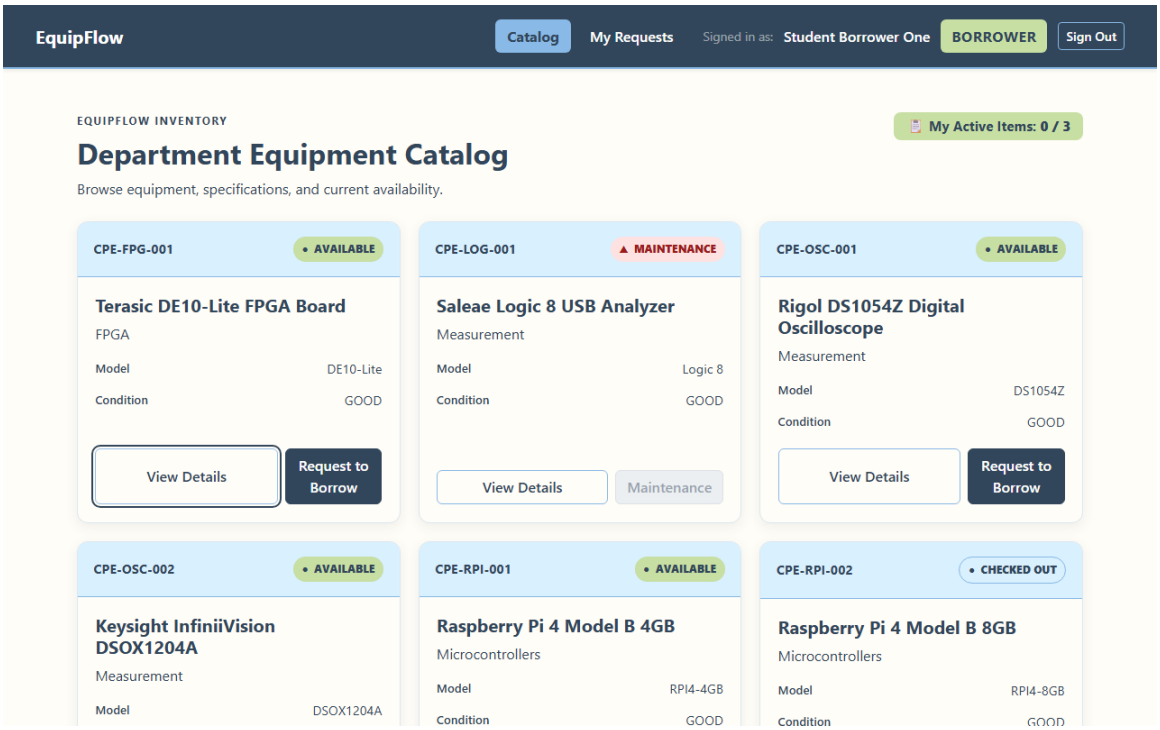
F-04 / E2E-01-01. The active-loan view retrieves the saved loan and due date. The test verifies the database row and a seven-day interval.



F-03 / AC-REQ-02 / E2E-01-02. An item becomes reserved after the modal is opened. The application rejects the request and displays an explanatory alert.



F-01 / AC-AUTH-06–07 / E2E-01-03. The Borrower remains outside the Custodian view. The API test provides the server-side 401/403 evidence.



E2E-01-04. Keyboard Tab exposes a visible focus indicator. The same test covers horizontal fit at mobile, tablet, and desktop widths.

F.2 · Build and automated test execution

Author evidence at source: Test execution log at 63e0426. The subsequent PR commit updated documents and screenshots; staging merged at e7ee384. The author log records 29 backend, 15 client and 4 browser passes. Build was reported in PR #16; that log captures the three test suites.

Command	Author run at 63e0426
npm test	29 passed / 0 failed / 0 skipped

Command	Author run at 63e0426
npm run test:client	15 passed / 0 failed / 0 skipped
npx playwright test	4 passed / 0 failed / 0 skipped

Author log excerpt

```
Tested PR Commit SHA: 63e04268449d9ca07b1b8c5a60a044c47854363d (63e0426)
 Test Files  4 passed (4)
      Tests  29 passed (29)
 Test Files  4 passed (4)
      Tests  15 passed (15)
  4 passed (13.3s)
```

Independent staging check

On staging e7ee384, build succeeded and all 29 backend and 15 client tests passed. The E2E run could not launch Chromium. The initial run had no browser executable, and the retry after installation failed at macOS browser startup before the assertions could run. This is separate from the author's 4/4 E2E run.

```
e7ee3849d617f2cb642e8e51e8fc807c91063f48
$ npm run build
□ built in 374ms
EXIT 0
$ npm test
Test Files  4 passed (4)
Tests  29 passed (29)
Duration  5.75s (tests 85%, import 10%, transform 4%)
EXIT 0
$ npm run test:client
Test Files  4 passed (4)
Tests  15 passed (15)
Duration  1.75s (environment 55%, tests 21%, transform 10%, setup 9%, import 4%)
EXIT 0
```

```
$ npm run test:e2e (retry)
browserType.launch: Target page, context or browser has been closed
macOS bootstrap_check_in: Permission denied (1100)
4 failed at browser startup; no E2E assertions verified
EXIT 1
```

AC-to-test results

The table reports the author run at 63e0426 for tests mapped in the Sprint 1 STS. See Part E for complete input, action and expected result. API-01-21 through API-01-23 run as one checkout test. Release results are pending.

Requirement / AC	Test ID and purpose	Automated test file	Result
FR-01 / AC-AUTH-01	API-01-01 · User login with valid email and password	server/tests/sprint-1/auth.api.test.ts	Pass
FR-01 / AC-AUTH-02	API-01-02 · User login with invalid password	server/tests/sprint-1/auth.api.test.ts	Pass
FR-01 / AC-AUTH-02	API-01-03 · User login with non-existent email	server/tests/sprint-1/auth.api.test.ts	Pass
FR-02 / AC-AUTH-03	API-01-04 · Fetch authenticated identity via GET /api/v1/auth/me	server/tests/sprint-1/auth.api.test.ts	Pass
FR-03 / AC-AUTH-04	API-01-05 · User logout via POST /api/v1/auth/logout	server/tests/sprint-1/auth.api.test.ts	Pass
FR-03 / AC-AUTH-04	API-01-06 · Request protected endpoint using revoked token	server/tests/sprint-1/auth.api.test.ts	Pass
FR-03 / AC-AUTH-05	API-01-07 · Two concurrent sessions for same user; logout Session 1	server/tests/sprint-1/auth.api.test.ts	Pass
FR-04 / AC-AUTH-06	API-01-08 · Request protected endpoint without Authorization header	server/tests/sprint-1/auth.api.test.ts	Pass

Requirement / AC	Test ID and purpose	Automated test file	Result
FR-04 / AC-AUTH-07	API-01-09 · Borrower attempts to access custodian endpoint	server/tests/sprint-1/auth.api.test.ts	Pass
NFR-01 / AC-AUTH-08	API-01-10 · Verify password hashing in database	server/tests/sprint-1/auth.api.test.ts	Pass
FR-05 / AC-CAT-01	API-01-11 · lists serialized catalog entries and current status for authenticated borrowers	server/tests/sprint-1/catalog.api.test.ts	Pass
FR-06 / AC-CAT-02	API-01-12 · returns complete equipment details to authenticated custodians	server/tests/sprint-1/catalog.api.test.ts	Pass
FR-06 / AC-CAT-02	API-01-13 · returns ITEM_NOT_FOUND for an unknown item ID	server/tests/sprint-1/catalog.api.test.ts	Pass
FR-09 / AC-REQ-01	API-01-14 · Borrower submits request for AVAILABLE item	server/tests/sprint-1/requests.api.test.ts	Pass
BR-01 / AC-REQ-02	API-01-15 · Borrower submits request for RESERVED or CHECKED_OUT item	server/tests/sprint-1/requests.api.test.ts	Pass
BR-04 / AC-REQ-03	API-01-16 · Borrower submits request when active quota >= 3	server/tests/sprint-1/requests.api.test.ts	Pass
BR-02 / AC-REQ-04	API-01-17 · Concurrent requests for the same available item	server/tests/sprint-1/requests.api.test.ts	Pass
FR-10 / AC-REQ-05	API-01-18 · Borrower queries personal requests via GET /api/v1/requests/my	server/tests/sprint-1/requests.api.test.ts	Pass
FR-11 / AC-REQ-06	API-01-19 · Cross-borrower data isolation check	server/tests/sprint-1/requests.api.test.ts	Pass
FR-13 / AC-CHK-01	API-01-20 · returns pending requests with borrower and item details, plus active loans	server/tests/sprint-1/checkout.api.test.ts	Pass
FR-14 / AC-CHK-02	API-01-21 · checks out a pending request with a seven-day due date and authenticated custodian attribution	server/tests/sprint-1/checkout.api.test.ts	Pass
BR-05 / AC-CHK-03	API-01-22 · Due date calculation check on checkout	server/tests/sprint-1/checkout.api.test.ts	Pass
BR-08 / AC-CHK-04	API-01-23 · Accountable custodian attribution check	server/tests/sprint-1/checkout.api.test.ts	Pass
BR-02 / AC-CHK-05	API-01-24 · rejects a duplicate checkout with CONCURRENT_CHECKOUT_CONFLICT	server/tests/sprint-1/checkout.api.test.ts	Pass
BR-02 / AC-CHK-06	API-01-25 · allows only one of two concurrent checkouts for the same request	server/tests/sprint-1/checkout.api.test.ts	Pass
BR-03 / AC-CHK-07	API-01-26 · forbids borrower checkout attempts	server/tests/sprint-1/checkout.api.test.ts	Pass
BR-04 / AC-REQ-03	API-01-27 · Concurrent requests from same borrower exceeding quota	server/tests/sprint-1/requests.api.test.ts	Pass
AC-AUTH-01	UI-01-01 · Login form rendering, credentials input & submit	client/tests/sprint-1/Login.test.tsx	Pass
AC-AUTH-02	UI-01-02 · Invalid credentials feedback banner display on 401	client/tests/sprint-1/Login.test.tsx	Pass
AC-CAT-01	UI-01-03 · renders catalog cards with readable status labels	client/tests/sprint-1/Catalog.test.tsx	Pass

Requirement / AC	Test ID and purpose	Automated test file	Result
AC-CAT-02	UI-01-04 · opens an accessible detail dialog with serial number and condition	client/tests/sprint-1/Catalog.test.tsx	Pass
AC-REQ-01	UI-01-05 · disables request button when item is unavailable	client/tests/sprint-1/Catalog.test.tsx	Pass
AC-REQ-03	UI-01-06 · disables request button when borrower active quota = 3	client/tests/sprint-1/Catalog.test.tsx	Pass
AC-REQ-05	UI-01-07 · renders active loans and pending requests with 7-day due date	client/tests/sprint-1/MyRequests.test.tsx	Pass
AC-CHK-01	UI-01-08 · renders the pending triage queue and active loans monitor	client/tests/sprint-1/CustodianDesk.test.tsx	Pass
AC-REQ-01, AC-CHK-02	E2E-01-01 · Full Domain Journey (Borrower Request to Custodian Checkout)	e2e/sprint-1/domain-journey.spec.ts	Pass
AC-REQ-02, AC-CHK-05	E2E-01-02 · Meaningful Business Rule Rejection (Duplicate Booking / Checkout & Quota)	e2e/sprint-1/rejection-rules.spec.ts	Pass
AC-AUTH-06, AC-AUTH-07	E2E-01-03 · Access Control & Role Segregation (Borrower Forbidden from Custodian Desk)	e2e/sprint-1/access-control.spec.ts	Pass
§4.1, §4.2	E2E-01-04 · Viewport Layout Audit (375px / 768px / 1280px) & Keyboard Focus Rings	e2e/sprint-1/layout-a11y.spec.ts	Pass

F.3 · Test excerpts and TDD

API-01-25 / AC-CHK-06 · exclusive checkout and one persisted loan · source lines

```
it('API-01-25: allows only one of two concurrent checkouts for the same request', async ()
   => {
  const { borrowRequest } = await createPendingRequest('RACE');
  const payload = { requestId: borrowRequest.id };
  const responses = await Promise.all([
    request(app).post('/api/v1/custodian/checkouts').set('Authorization', `Bearer
  ${custodianToken}`).send(payload),
    request(app).post('/api/v1/custodian/checkouts').set('Authorization', `Bearer
  ${custodianToken}`).send(payload),
  ]);

  expect(responses.map(({ status }) => status).sort()).toEqual([201, 409]);
  const success = responses.find(({ status }) => status === 201)!;
  const conflict = responses.find(({ status }) => status === 409)!;
  loanIds.push(success.body.data.id);
  expect(conflict.body.error.code).toBe('CONCURRENT_CHECKOUT_CONFLICT');
  expect(await prisma.loan.count({ where: { requestId: borrowRequest.id } })).toBe(1);
});
```

API-01-26 / AC-CHK-07 · authenticated Borrower is forbidden · source lines

```
it('API-01-26: forbids borrower checkout attempts', async () => {
  const { borrowRequest } = await createPendingRequest('ROLE');
  const response = await request(app)
    .post('/api/v1/custodian/checkouts')
    .set('Authorization', `Bearer ${borrowerToken}`)
    .send({ requestId: borrowRequest.id });

  expect(response.status).toBe(403);
  expect(response.body.error.code).toBe('FORBIDDEN_ROLE');
});
```

Both cases are in the passing checkout suite in the author log.

Test-first history

Version	Evidence
890f66e	13 Nov · test-first commit before feature implementation. Reproduction: 5 checkout tests failed, including API-01-25 (actual 404/404 vs expected 201/409) and API-01-26 (actual 404 vs expected 403).
017b48e	13 Nov · checkout implementation follows the test commit.
5794369	14 Nov · dedicated borrower fixture addresses the reviewer's cross-suite quota collision. Author reports 29/29 server and 15/15 client; reviewer verified the fix.

Contract/test design appeared on 12 Nov in 0d2d17d before the feature commits. The checkout file passed all five tests on e7ee384 in the independent staging run. The red transcript was reproduced at the historical commit, rather than captured during the original implementation.

F.4 · Peer review and integration

Work and branch	Author / reviewer	Substantive review	Integration
F-04 · GitHub #7 / PR #15 feat/f04-custodian-checkout	itspxsh / NxNxmm	Requested test isolation; fix verified	Staging f098b1e; main release pending
Integration / E2E-01-01 to 04 · GitHub #8 / PR #16 test/sprint1-verification	NxNxmm / itspxsh	Requested fixture and log corrections; approved 9249e88	Staging e7ee384; main release pending

Board at report intake: checkout Issue #7 In review with PR #15 merged; verification Issue #8 In progress with PR #16 merged; report Issue #9 Backlog at intake. Project board. Update this snapshot for the submitted PDF.

Contribution summary

Contributor	Sprint 1 work evidenced in Git
itspxsh	Scaffold (PR #11), catalog (PR #13), checkout (PR #15), review of teammate's verification (PR #16), and compilation of this report.
NxNxmm	Baseline (PR #1), engineering contracts (PR #10), authentication (PR #12), borrowing (PR #14), verification (PR #16), and review of checkout (PR #15).

AI use. Codex assisted with compiling, formatting and cross-checking this draft against repository sources. Each implementation result and review claim is tied to recorded evidence. Teammate to confirm any additional AI assistance before submission.

F.5 · Reproduction and open items

From a clean checkout, use the Node/npm versions compatible with the lockfile, install dependencies, copy .env.example privately, set a local JWT secret, generate Prisma Client, migrate and seed SQLite, then start the client and API. The client runs on localhost:5173 and the API on localhost:3000. Install the matching Playwright Chromium before the E2E run; use only synthetic local seed data.

```
npm ci
npm run db:generate
npm run db:migrate
npm run db:seed
npm run dev
npm run build
npm test
npm run test:client
npm run test:e2e
```

Open before submission: record the release PR to main, immutable tag, exact release SHA and build/test outcomes; confirm teammate cover details and declaration; refresh board and review evidence. Inventory

returns, overdue processing and production deployment are later checkpoints. Some baseline design/tool versions are planning values; reproduced commands and results apply to the lockfile of the checked commit.